symmetry

MDPI

Article

Self-Adapting CPU Scheduling for Mixed Database Workloads via Hierarchical Deep Reinforcement Learning

Suchuan Xing [1], Yihan Wang [2,*] and Wenhe Liu [3]

[1] Department of Electrical and Computer Engineering, Duke University, Durham, NC 27708, USA; sx80@alumni.duke.edu
[2] School of Engineering and Applied Science, The University of Pennsylvania, Philadelphia, PA 19104, USA
[3] School of Computer Science, Carnegie Mellon University, Pittsburgh, PA 15213, USA; wenhel@cs.cmu.edu
* Correspondence: wangyh1@alumni.upenn.edu

Abstract

Modern database systems require autonomous CPU scheduling frameworks that dynamically optimize resource allocation across heterogeneous workloads while maintaining strict performance guarantees. We present a novel hierarchical deep reinforcement learning framework augmented with graph neural networks to address CPU scheduling challenges in mixed database environments comprising Online Transaction Processing (OLTP), Online Analytical Processing (OLAP), vector processing, and background maintenance workloads. Our approach introduces three key innovations: first, a symmetric two-tier control architecture where a meta-controller allocates CPU budgets across workload categories using policy gradient methods while specialized sub-controllers optimize process-level resource allocation through continuous action spaces; second, graph neural network-based dependency modeling that captures complex inter-process relationships and communication patterns while preserving inherent symmetries in database architectures; and third, meta-learning integration with curiosity-driven exploration enabling rapid adaptation to previously unseen workload patterns without extensive retraining. The framework incorporates a multi-objective reward function balancing Service Level Objective (SLO) adherence, resource efficiency, symmetric fairness metrics, and system stability. Experimental evaluation through high-fidelity digital twin simulation and production deployment demonstrates substantial performance improvements: 43.5% reduction in p99 latency violations for OLTP workloads and 27.6% improvement in overall CPU utilization, with successful scaling to 10,000 concurrent processes maintaining sub-3% scheduling overhead. This work represents a significant advancement toward truly autonomous database resource management, establishing a foundation for next-generation self-optimizing database systems with implications extending to broader orchestration challenges in cloud-native architectures.

Keywords: hierarchical deep reinforcement learning; database systems; CPU scheduling; resource management; graph symmetry; symmetric fairness

check for updates

Academic Editor: Zhixun Su

Received: 9 June 2025
Revised: 4 July 2025
Accepted: 6 July 2025
Published: 10 July 2025

Citation: Xing, S.; Wang, Y.; Liu, W. Self-Adapting CPU Scheduling for Mixed Database Workloads via Hierarchical Deep Reinforcement Learning. *Symmetry* **2025**, *17*, 1109. https://doi.org/10.3390/sym17071109

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Modern database systems confront an increasingly complex landscape of computational workloads, ranging from traditional Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP) to emerging paradigms such as vector databases and Hybrid Transactional/Analytical Processing (HTAP). This heterogeneity presents fundamental challenges for resource management, particularly in Central Processing Unit

(CPU) scheduling, where static allocation strategies fail to adapt to dynamic workload characteristics and competing Service Level Objectives (SLOs) [1,2]. The proliferation of cloud-native architectures further amplifies these challenges, as multi-tenant environments demand efficient resource sharing while maintaining strict performance guarantees for diverse applications.

Traditional CPU scheduling methods in database systems rely on predetermined policies and heuristic-based algorithms that struggle to optimize across multiple dimensions simultaneously. While systems like Oracle's Database Resource Manager provide basic workload prioritization through resource consumer groups, they require extensive manual tuning and cannot dynamically adapt to changing workload patterns [3]. The fundamental challenge lies in achieving symmetric fairness across heterogeneous workloads while maintaining optimal resource utilization—a problem that inherently involves balancing competing symmetries in resource allocation, process prioritization, and performance optimization. Recent advances in machine learning, particularly deep reinforcement learning (DRL), offer promising alternatives by learning optimal scheduling policies through interaction with the environment [4,5]. However, existing DRL methods face significant limitations when applied to database CPU scheduling: they typically model the problem as a monolithic optimization task, failing to capture the hierarchical nature of resource allocation decisions and the complex interdependencies between database processes. Moreover, they often overlook the inherent symmetric properties of database workloads, such as symmetric communication patterns in distributed query processing, symmetric load distribution requirements across parallel workers, and the need for symmetric fairness in multi-tenant environments.

The emergence of graph neural networks (GNNs) presents new opportunities for modeling structural relationships in distributed systems. Recent work demonstrates that GNNs can effectively capture dependencies in resource allocation problems, achieving significant performance improvements in scheduling tasks [6,7]. Crucially, GNNs are inherently designed to respect graph symmetries through permutation invariance, making them naturally suited for modeling the symmetric communication patterns and symmetric process relationships that characterize database systems. Similarly, hierarchical reinforcement learning has shown success in decomposing complex control problems into manageable sub-tasks, enabling more efficient learning and better generalization [8,9]. Meta-learning methods further enhance adaptability by enabling rapid adjustment to new workload patterns without extensive retraining [10,11].

Despite these advances, no existing work successfully integrates these complementary technologies to address the unique challenges of CPU scheduling in heterogeneous database environments. Current methods suffer from three critical limitations. First, they lack the architectural sophistication to handle multi-level scheduling decisions, from coarse-grained workload category allocation to fine-grained process-level optimization. Second, they fail to model the rich structural dependencies between database processes, missing opportunities for context-aware scheduling that considers communication patterns and resource contention. Third, they cannot rapidly adapt to previously unseen workload combinations, a crucial requirement in production environments where workload patterns evolve continuously.

To address these limitations, this paper proposes a novel self-adaptive CPU scheduling framework that leverages hierarchical deep reinforcement learning (HDRL) augmented with GNNs. The proposed method introduces a two-tier architecture where a high-level meta-controller allocates CPU budgets across workload categories using policy gradient methods, while low-level sub-controllers fine-tune process-specific resource controls through continuous action spaces. This hierarchical decomposition naturally aligns with

the structure of database workloads and enables more efficient learning by reducing the complexity faced by each agent. This paper employs GNNs to learn structural embeddings of database process interactions, capturing dependencies that traditional methods overlook. The framework incorporates curiosity-driven exploration [12] and meta-learning techniques to ensure rapid adaptation to novel workload patterns, addressing the critical challenge of generalization in production deployments.

The remainder of this paper is organized as follows. Section 2 reviews related work in database resource management, DRL for systems, and hierarchical learning methods. Section 3 presents our system architecture and the HDRL framework design. Section 4 details the GNN-based dependency modeling and state representation. Section 5 describes the reward engineering and training methodology. Section 6 presents experimental evaluation. Section 7 concludes the paper.

2. Related Work

Our work builds upon advances in several research areas: deep reinforcement learning for resource allocation, HDRL, GNNs for system optimization, database resource management, and meta-learning methods. We examine each area and highlight how our approach addresses existing limitations.

2.1. Deep Reinforcement Learning for Resource Allocation

Deep reinforcement learning has emerged as a powerful paradigm for dynamic resource allocation in computer systems [13,14]. DeepRM [4] pioneered the application of DRL to cluster resource management, demonstrating that neural networks can learn effective scheduling policies without domain-specific heuristics. The system uses a policy gradient approach to handle variable job sizes and cluster configurations, achieving significant improvements over traditional schedulers. However, DeepRM focuses on batch job scheduling rather than the continuous, latency-sensitive workloads characteristic of database systems.

Recent work has extended DRL to cloud environments with greater complexity. Feng et al. [15] provide a comprehensive survey of DRL methods for cloud resource scheduling, identifying key challenges including high-dimensional state spaces and multi-objective optimization. ACRA [16] employs Actor-Critic methods for adaptive resource allocation in cloud datacenters, achieving improved efficiency over static policies. Similarly, Dong et al. [17] apply DRL to workflow scheduling, demonstrating the benefits of learning-based methods for handling dynamic workload patterns. However, these systems treat resource allocation as a flat optimization problem, missing opportunities for hierarchical decomposition that could improve learning efficiency and policy interpretability.

The application of DRL to database-specific resource management has received limited attention. CDBTune [18] uses DRL for database configuration tuning but focuses on static parameter optimization rather than dynamic resource allocation. While effective for its intended purpose, CDBTune cannot adapt to rapidly changing workload patterns or manage fine-grained CPU scheduling decisions. Our work extends these concepts to dynamic CPU scheduling with hierarchical control structures specifically designed for heterogeneous database workloads.

2.2. Hierarchical Reinforcement Learning in Systems

Hierarchical reinforcement learning decomposes complex tasks into manageable subtasks, enabling more efficient learning and better generalization. The comprehensive survey by Pateria et al. [8] identifies key HRL architectures and their applications across domains. In systems contexts, Liu et al. [5] demonstrate a two-level framework for cloud resource

allocation, where high-level decisions about virtual machine placement inform low-level power management. Their approach achieves significant energy savings but does not address the unique challenges of database workload heterogeneity.

Recent advances in HRL have focused on automatic hierarchy discovery and causality-driven decomposition. Hu et al. [9] propose learning hierarchical structures through causal analysis of state-action relationships, improving sample efficiency and interpretability. Manczak et al. [19] apply HRL to power network control, demonstrating superior performance in handling multi-timescale decisions. However, these methods have not been adapted to the specific requirements of database CPU scheduling, where workload categories exhibit distinct performance characteristics and SLO requirements.

The integration of HRL with other learning paradigms remains underexplored. While Li et al. [20] combine HRL with auxiliary rewards to improve exploration, and Nachum et al. [21] address data efficiency challenges, no existing work combines HRL with GNNs for modeling system dependencies. Our work fills this gap by using GNNs to capture inter-process relationships within the hierarchical control framework.

2.3. GNNs for System Optimization

GNNs have demonstrated remarkable success in modeling relational structures within distributed systems. Zhao et al. [6] introduce GNN-based distributed scheduling, showing that graph representations can capture complex dependencies between tasks and resources. The approach achieves near-optimal scheduling decisions while maintaining computational efficiency. Building on this foundation, Zhao et al. [22] extend GNN scheduling to optimize for delay-oriented objectives, demonstrating the flexibility of graph-based methods.

Recent work has applied GNNs to database-specific challenges. XGNN [7] accelerates multi-GPU GNN training through intelligent memory management, while Wang et al. [23] use GNNs for distributed query optimization. These systems demonstrate the value of modeling structural dependencies but focus on specific optimization problems rather than general resource management. FlexGraph [24] provides a flexible framework for distributed GNN training but does not address the integration with reinforcement learning for dynamic decision-making.

The combination of GNNs with reinforcement learning for systems optimization remains nascent. While Song et al. [25] propose GNN-based control for networked systems, the approach focuses on continuous control rather than discrete scheduling decisions. Our work pioneers the integration of GNNs within a hierarchical reinforcement learning framework, using graph representations to inform both high-level and low-level scheduling decisions.

2.4. Database Resource Management and Scheduling

Traditional database resource management relies on rule-based policies and static configurations. Oracle's Resource Manager and similar systems provide basic workload isolation but require extensive manual tuning. Recent research has focused on adaptive methods that respond to changing workload characteristics. Yogatama et al. [3] propose intelligent data placement and query routing for heterogeneous CPU-GPU systems, achieving significant performance improvements through workload-aware scheduling. However, their approach uses analytical models rather than learning-based methods, limiting adaptability to unforeseen workload patterns.

The challenge of scheduling heterogeneous database workloads has received increasing attention. Nogueira et al. [26] address workload placement optimization, while Lyu et al. [27] provide fine-grained modeling for resource management in big data processing. These systems demonstrate the complexity of modern database workloads but rely on

traditional optimization techniques. HetExchange [2] encapsulates CPU-GPU parallelism in JIT-compiled engines, showing the benefits of heterogeneous processing, but does not address dynamic resource allocation between competing workloads.

Multi-tenant database systems present additional scheduling challenges. El et al. [28] leverage workload prediction for query optimization in parallel DBMSs, demonstrating the value of anticipating resource needs. However, their approach focuses on query-level optimization rather than system-wide resource allocation. Our work addresses this gap by providing hierarchical control that spans from workload categories to individual processes.

2.5. Autonomous Database Management Systems

The vision of autonomous database management has driven significant research in self-tuning and self-managing systems. NeurDB [1] presents a comprehensive AI-powered autonomous data system that integrates multiple learning components for self-optimization. PilotScope [29] provides a framework for steering databases with machine learning drivers, enabling systematic integration of ML models into database operations. These systems demonstrate the potential for AI-driven database management but focus primarily on configuration tuning and query optimization rather than resource scheduling.

Recent work has emphasized the importance of efficient learning in autonomous systems. Zhang et al. [30] propose transfer learning for database configuration, reducing the time required to adapt to new workloads. The Holon approach [31] simultaneously tunes multiple components, addressing the challenge of component interactions. Lim et al. [32] accelerate behavior model training through efficient query execution sampling. While these advances improve the practicality of autonomous systems, they do not address the specific challenges of real-time CPU scheduling with strict SLO requirements.

The integration of autonomous techniques with production systems remains challenging. Zhu et al. [33] describe efforts to build autonomous data services on Azure, highlighting the importance of safe deployment and gradual rollout. Their experiences inform our design of shadow mode evaluation and A/B testing frameworks, ensuring that learned policies can be safely validated before production deployment.

2.6. Meta-Learning and Curiosity-Driven Exploration

Meta-learning enables rapid adaptation to new tasks by learning to learn from prior experience. Nagabandi et al. [10] demonstrate meta-reinforcement learning for real-world adaptation, showing that agents can quickly adjust to environmental changes. In systems contexts, Beck et al. [34] survey meta-reinforcement learning methods, identifying opportunities for application to resource management. However, existing work has not explored meta-learning for database workload adaptation, where the distribution of workload patterns shifts continuously.

Curiosity-driven exploration addresses the challenge of discovering effective policies in complex environments. Alet et al. [11] combine meta-learning with curiosity algorithms, enabling agents to actively seek informative experiences. Jarrett et al. [12] introduce curiosity in hindsight for stochastic environments, improving exploration efficiency in partially observable settings. Aubret et al. [35] provide an information-theoretic perspective on intrinsic motivation, offering principled methods to exploration.

The application of these advanced learning paradigms to systems optimization remains limited. While Yuan et al. [36] explore intrinsically motivated RL and Raileanu et al. [37] propose impact-driven exploration, these techniques have not been adapted to the specific challenges of database resource management. Our work integrates curiosity-driven exploration within the hierarchical framework, encouraging the discovery of effective

scheduling policies for previously unseen workload combinations. The main contributions are as follows:

- This work presents the first HDRL framework for database CPU scheduling that decomposes the problem into hierarchical control levels, enabling efficient learning and deployment in complex heterogeneous environments.
- The proposed method incorporates GNNs to explicitly model inter-process dependencies. This enables the generation of context-aware scheduling decisions that effectively account for intricate communication patterns and resource contention among database processes.
- The framework employs a multi-objective reward function that balances strict SLO adherence with resource efficiency, incorporating novel penalties for oscillatory behavior and fairness metrics to prevent workload starvation.
- Extensive empirical evaluation demonstrates the efficacy of the proposed methodology. For OLTP workloads, the proposed method achieves a notable reduction in p99 latency violations of up to 43.5%, concurrently improving overall CPU utilization by 27.6% when benchmarked against contemporary state-of-the-art schedulers.

3. System Architecture and HDRL Framework Design

3.1. Overview

We present a hierarchical deep reinforcement learning framework for adaptive CPU scheduling in heterogeneous database environments. The system comprises three main components: a two-tier HDRL control structure with meta-controller and sub-controllers, a GNN-based state representation module that captures inter-process dependencies, and a monitoring and actuation layer that interfaces with the operating system's control groups (cgroups) mechanism. Figure 1 illustrates the overall architecture.

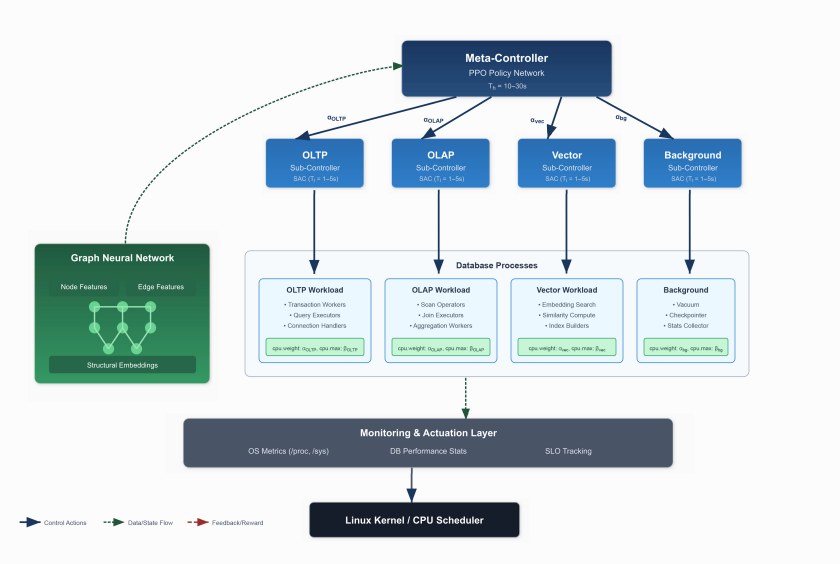


Figure 1. Comprehensive system architecture of the hierarchical deep reinforcement learning framework for autonomous database CPU scheduling. The architecture comprises three integrated components: (1) Hierarchical Control Structure: The meta-controller (top tier) makes strategic CPU budget allocation decisions across four workload categories—OLTP, OLAP, Vector, and Background—using policy gradient methods with observation period T_h. (2) Graph Neural Network Module: Captures complex inter-process dependencies through dynamic graph construction, where nodes represent database processes and edges encode communication patterns, resource contention, and workflow dependencies. (3) Monitoring and Actuation Layer: Interfaces with the Linux kernel control groups (cgroups) v2 mechanism to enforce CPU scheduling decisions through cpu.weight (proportional sharing) and cpu.max (bandwidth limiting) parameters.

The framework operates at multiple timescales to balance responsiveness with stability. The meta-controller makes strategic decisions every T_h seconds, where T_h typically ranges from 10 to 30 s, allocating CPU budgets across workload categories. Sub-controllers operate at finer granularity with period T_l, typically between 1 and 5 s, optimizing resource allocation within their assigned budgets. This temporal hierarchy naturally aligns with database workload characteristics, where aggregate patterns evolve slowly while individual query requirements fluctuate rapidly.

3.2. Hierarchical Control Structure

3.2.1. Meta-Controller Design

The meta-controller serves as the high-level decision maker, responsible for strategic CPU allocation across workload categories. It observes aggregate performance metrics and determines the optimal distribution of computational resources among OLTP, OLAP, vector processing, and background maintenance workloads. The meta-controller operates in a Markov Decision Process (MDP) defined by the tuple $\mathcal{M}_h = (\mathcal{S}_h, \mathcal{A}_h, \mathcal{P}_h, \mathcal{R}_h, \gamma_h)$. In this formulation, $\mathcal{S}_h$ represents the high-level state space comprising aggregate workload metrics that capture the overall system behavior. The action space $\mathcal{A}_h$ defines possible CPU budget allocations across categories, while $\mathcal{P}_h$ captures state transition probabilities that model how the system evolves in response to actions. The reward function $\mathcal{R}_h$ encodes our multi-objective optimization goals, and the discount factor γ_h controls the balance between immediate and long-term rewards.

The state representation for the meta-controller is defined as follows:

$$s_h = \{m_{OLTP}, m_{OLAP}, m_{vec}, m_{bg}, u_{global}, v_{SLO}\} \tag{1}$$

where m_{OLTP}, m_{OLAP}, m_{vec}, and m_{bg} represent comprehensive performance metrics for OLTP, OLAP, vector processing, and background maintenance workloads, respectively. Each metric vector m_i includes CPU utilization, memory consumption, I/O statistics, and workload-specific indicators such as transaction rates or query completion times. The global utilization metric u_{global} captures overall system CPU usage as a percentage, while v_{SLO} represents a vector of SLO violation statistics including violation counts, severity, and duration for each workload category.

Actions taken by the meta-controller specify CPU budget percentages:

$$a_h = \{\alpha_{OLTP}, \alpha_{OLAP}, \alpha_{vec}, \alpha_{bg}\} \tag{2}$$

subject to the constraint $\sum_i \alpha_i = 1$ and $\alpha_i \geq \alpha_{min}$, where α_{min} is a minimum allocation threshold (typically 0.05) that prevents starvation of any workload category. Each α_i represents the fraction of total CPU resources allocated to workload category i.

We employ proximal policy optimization (PPO) [38] for the meta-controller due to its stability and sample efficiency in policy gradient methods. The policy network $\pi_\theta(a_h|s_h)$ outputs a categorical distribution over discretized budget allocations, enabling stochastic exploration during training while maintaining deterministic execution in production. The value network $V_\phi(s_h)$ estimates expected returns for variance reduction in policy gradient updates.

3.2.2. Sub-Controller Architecture

Each workload category maintains a dedicated sub-controller that operates within its allocated budget from the meta-controller. These sub-controllers make fine-grained decisions about CPU shares and bandwidth limits for individual processes or process groups within their category. This design enables specialized optimization strategies tailored to

specific workload characteristics, such as prioritizing latency for OLTP workloads while maximizing throughput for OLAP queries.

The sub-controller for category c operates in its own MDP $\mathcal{M}_l^c = (\mathcal{S}_l^c, \mathcal{A}_l^c, \mathcal{P}_l^c, \mathcal{R}_l^c, \gamma_l)$. The state space $\mathcal{S}_l^c$ contains detailed process-level metrics and inter-process dependencies specific to category c. Actions $\mathcal{A}_l^c$ specify cgroup parameters for processes in the category, including CPU shares and bandwidth limits. The reward function $\mathcal{R}_l^c$ is derived from category-specific performance objectives, such as minimizing p99 latency for OLTP or maximizing query throughput for OLAP.

For continuous control over CPU shares, we implement sub-controllers using Soft Actor-Critic (SAC) [39], which provides stable learning in continuous action spaces through entropy regularization. The actor network outputs parameters for a squashed Gaussian distribution:

$$a_l^c = \tanh(\mu_\psi(s_l^c) + \sigma_\psi(s_l^c) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I) \tag{3}$$

where $\mu_\psi(s_l^c)$ represents the mean action computed by the actor network with parameters ψ, $\sigma_\psi(s_l^c)$ is the learned standard deviation for exploration, $\odot$ denotes element-wise multiplication, and ϵ is Gaussian noise. The tanh function bounds actions to the range $[-1, 1]$, which are then mapped to valid cgroup parameters.

3.3. State Representation and Feature Engineering

Effective state representation is crucial for learning optimal policies. We design features that capture both instantaneous performance and temporal dynamics across multiple granularities. For each database process p, we extract comprehensive metrics including CPU utilization u_p^{cpu}, calculated as the ratio of CPU time consumed to wall clock time elapsed. Memory pressure m_p is computed as the ratio of resident set size (RSS) to the configured memory limit, indicating potential memory bottlenecks. I/O statistics io_p comprise read rate, write rate, and I/O wait time percentages. Database-specific metrics db_p include active session counts, wait event distributions, and p99 latency measurements.

These raw features undergo normalization using exponential moving averages to produce stable inputs for the neural networks. The normalization process ensures that features with different scales contribute equally to the learning process while preserving temporal patterns essential for decision-making.

Aggregate features for workload category c are represented as follows:

$$m_c = \{\bar{u}_c^{cpu}, \text{VAR}(u_c^{cpu}), \bar{l}_c, l_c^{p99}, q_c, v_c^{SLO}\} \tag{4}$$

where $\bar{u}_c^{cpu}$ denotes the mean CPU utilization across all processes in category c, $\text{VAR}(u_c^{cpu})$ captures utilization variance indicating workload stability, $\bar{l}_c$ represents average query latency, l_c^{p99} tracks the 99th percentile latency critical for SLO compliance, q_c measures queue length or pending request count, and v_c^{SLO} quantifies SLO violations in the recent time window.

Temporal context is incorporated through exponentially weighted moving averages (EWMA) and rate calculations:

$$f_t^{ewma} = \beta f_{t-1}^{ewma} + (1 - \beta) f_t \tag{5}$$

$$f_t^{rate} = \frac{f_t - f_{t-\Delta t}}{\Delta t} \tag{6}$$

In these equations, f_t represents any feature value at time t, β is the smoothing factor controlling the weight of historical values, and Δt is the time interval for rate calcula-

tion. Multiple timescales with $\beta \in \{0.9, 0.99, 0.999\}$ capture short-term fluctuations and long-term trends, enabling the model to respond to both transient spikes and sustained workload changes.

3.4. Integration with Operating System Mechanisms

3.4.1. Control Group Interface

Our framework actuates scheduling decisions through Linux control groups v2, which provide fine-grained CPU resource control at the process group level. The actuation layer translates high-level actions from the RL agents into specific cgroup configurations. The primary control mechanisms include `cpu.weight` for specifying relative CPU shares in proportional scheduling, `cpu.max` for enforcing hard bandwidth limits through quota and period settings, and `cpuset.cpus` for CPU affinity assignments that optimize NUMA locality.

The translation function $\tau : \mathcal{A} \to \mathcal{C}$ maps RL actions to cgroup parameters:

$$\tau(a) = \{weight_i = \lfloor \alpha_i \cdot W_{base} \rfloor, quota_i = \lfloor \beta_i \cdot Q_{max} \rfloor\} \tag{7}$$

where α_i represents the normalized action value for process group i, W_{base} is the base weight value (typically 100 for standard configurations), β_i is the bandwidth allocation factor, and Q_{max} denotes the maximum quota value determined by the system's CPU capacity and scheduling period.

3.4.2. Monitoring Infrastructure

Continuous monitoring provides the state observations required for informed decision-making. Our multi-source monitoring pipeline combines OS-level metrics extracted via `/proc` and `/sys` filesystem interfaces, database performance statistics obtained through native database APIs and performance schemas, and application-level instrumentation for precise SLO tracking. The monitoring subsystem maintains circular buffers for efficient feature computation, implementing a sliding window approach that balances memory usage with historical context preservation. To minimize overhead, the system employs batched collection strategies and asynchronous processing pipelines that decouple metric gathering from feature computation.

3.5. Safety Mechanisms and Deployment Considerations

3.5.1. Action Safety Constraints

System stability is paramount in production database environments. We enforce comprehensive safety constraints on all actions through the safety function

$$a_{safe} = \text{clip}(a_{raw}, a_{min}, a_{max}) \cdot \mathbb{1}[\Delta a < \delta_{max}] \tag{8}$$

where a_{raw} represents the raw action output from the neural network, a_{min} and a_{max} define the permissible action range based on system capacity and workload requirements, Δa measures the change from the previous action, δ_{max} limits the maximum rate of change to prevent oscillations, and $\mathbb{1}[\cdot]$ is an indicator function that enforces the rate constraint.

3.5.2. Fallback Mechanisms

The system maintains robust fallback policies for critical scenarios to ensure continuous operation even under unexpected conditions. When SLO violations exceed predefined thresholds, the system triggers immediate priority boosts for affected workloads, temporarily overriding learned policies to restore service quality. In cases of resource exhaustion where total demand exceeds capacity, the framework activates a conservative equal-share allocation strategy that ensures fair resource distribution while preventing system overload.

When model uncertainty, measured through prediction variance or ensemble disagreement, exceeds confidence threshold ϵ_{conf}, the system gracefully reverts to a baseline policy derived from historical best practices.

3.5.3. Shadow Mode Operation

Initial deployment follows a cautious shadow mode approach where RL agents observe production systems and learn optimal policies without actuating decisions. During this phase, agent predictions are continuously logged for offline evaluation against actual scheduling decisions. Divergence metrics quantify policy differences from the production scheduler, enabling thorough validation before transitioning to active control. This shadow mode operation typically continues for several weeks, accumulating sufficient evidence of policy effectiveness across diverse workload conditions.

3.6. Training Architecture

The distributed training architecture separates policy learning from execution to minimize production impact while enabling continuous improvement. Training workers consume experience tuples from a centralized replay buffer populated by production observations, implementing an asynchronous learning paradigm that scales with system size. We employ prioritized experience replay [40] to focus learning on critical state transitions, particularly those involving SLO violations or significant performance changes. Priority scores are computed based on temporal difference errors and the importance of maintaining SLO compliance.

The meta-controller and sub-controllers train asynchronously with periodic synchronization to maintain policy consistency. This asynchronous training prevents bottlenecks while ensuring that the hierarchical structure remains coherent. Hyperparameter optimization employs Bayesian optimization techniques over a held-out validation set of historical workload traces, systematically exploring the hyperparameter space to identify configurations that generalize well across diverse workload patterns. The optimization process considers multiple objectives including convergence speed, final performance, and stability under varying conditions.

4. GNN-Based Dependency Modeling and State Representation

Database workloads exhibit complex interdependencies that traditional feature vectors fail to capture adequately. Processes communicate through shared memory segments, compete for buffer pool pages, and coordinate through locks and latches. These interactions form a dynamic graph structure where nodes represent processes and edges encode relationships. While conventional methods treat each process independently, losing critical relational information, we leverage GNNs [41] to learn representations that explicitly model these structural dependencies while preserving the fundamental symmetries inherent in database architectures.

The graph-based method offers several advantages for CPU scheduling from a symmetry perspective. First, it naturally captures transitive dependencies through symmetric message propagation—if process p_1 waits for p_2, and p_2 waits for p_3, the GNN can propagate this information symmetrically to inform scheduling decisions for all three processes. Second, the learned embeddings incorporate neighborhood context through permutation-invariant aggregation functions, enabling the scheduler to anticipate resource contention before it manifests as performance degradation. Third, the method generalizes to varying numbers of processes without architectural modifications, as GNNs inherently handle graphs of different sizes through symmetric pooling operations that respect the invariance properties essential for scalable database systems.

4.1. Dynamic Graph Construction

We construct a directed graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ at each time step t, where $\mathcal{V}_t$ represents the set of active database processes and $\mathcal{E}_t$ captures their interactions. The graph evolves dynamically as processes spawn, terminate, and change their communication patterns.

4.1.1. Node Features

Each node $v_i \in \mathcal{V}_t$ corresponds to a database process with feature vector $\mathbf{x}_i \in \mathbb{R}^{d_v}$, where d_v denotes the node feature dimensionality. The feature vector comprises four components:

$$\mathbf{x}_i = [\mathbf{r}_i; \mathbf{p}_i; \mathbf{w}_i; \mathbf{h}_i] \tag{9}$$

where $\mathbf{r}_i \in \mathbb{R}^{d_r}$ denotes resource utilization metrics, $\mathbf{p}_i \in \mathbb{R}^{d_p}$ represents performance indicators, $\mathbf{w}_i \in \mathbb{R}^{d_w}$ encodes workload-specific attributes, $\mathbf{h}_i \in \mathbb{R}^{d_h}$ captures historical features, and the semicolon notation $[\cdot;\cdot]$ indicates vector concatenation. The total feature dimension satisfies $d_v = d_r + d_p + d_w + d_h$.

Resource utilization metrics $\mathbf{r}_i$ include instantaneous CPU usage percentage normalized to $[0, 1]$, memory allocation relative to configured limits, I/O bandwidth consumption in MB/s, and network traffic rates. Performance indicators $\mathbf{p}_i$ encompass query execution time in milliseconds, transaction throughput measured in operations per second, lock wait duration, and buffer hit ratios expressed as percentages. Workload attributes $\mathbf{w}_i$ encode the process type through one-hot encoding for OLTP, OLAP, vector, or background categories, operation category distinguishing read, write, or mixed workloads, and priority levels assigned by the database engine ranging from 1 to 10. Historical features $\mathbf{h}_i$ maintain exponentially weighted moving averages of key metrics across multiple timescales, specifically using decay factors $\beta \in \{0.9, 0.99, 0.999\}$ to capture short-term fluctuations and long-term trends.

4.1.2. Edge Construction and Features

Edges in $\mathcal{E}_t$ represent various forms of inter-process dependencies discovered through system monitoring. Communication edges connect processes that exchange data through shared memory, sockets, or message queues, with the monitoring layer tracking inter-process communication calls to construct edges weighted by communication frequency measured in messages per second and data volume in kilobytes. Resource contention edges link processes competing for the same resources, detected through lock wait chains, buffer pool page conflicts, and CPU run queue analysis, with edge weights reflecting contention severity measured by cumulative wait times in milliseconds and conflict frequency per minute. Workflow edges capture logical dependencies in multi-stage query execution, where the database query planner provides execution graphs that map to process-level dependencies, ensuring upstream operators receive appropriate resources before downstream stages. Temporal edges connect the same process across consecutive time steps through self-loops that carry information about process state evolution, helping predict future resource requirements based on historical patterns.

Each edge $(i, j) \in \mathcal{E}_t$ from process i to process j carries feature vector $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$ with dimensionality d_e:

$$\mathbf{e}_{ij} = [w_{ij}^{comm}, w_{ij}^{cont}, w_{ij}^{dep}, \tau_{ij}] \tag{10}$$

where $w_{ij}^{comm} \in \mathbb{R}$ quantifies communication intensity normalized by maximum observed bandwidth, $w_{ij}^{cont} \in \mathbb{R}$ measures resource contention severity on a logarithmic scale, $w_{ij}^{dep} \in \{0, 1\}$ indicates presence of workflow dependency, and $\tau_{ij} \in [0, 1]$ represents temporal persistence computed as the fraction of recent time windows where the edge existed.

4.2. Graph Neural Network Architecture

We employ a message passing neural network (MPNN) [42] architecture that iteratively refines node representations by aggregating information from neighboring nodes. The architecture consists of K graph convolution layers followed by hierarchical pooling operations to generate both graph-level and category-specific embeddings.

4.2.1. Message Passing Layers

The MPNN performs K iterations of message passing, where each layer $k \in \{1, \ldots, K\}$ updates node representations through neighborhood aggregation. The message passing operation for node i at layer k computes messages from all neighboring nodes:

$$\mathbf{m}_i^{(k)} = \text{AGG}_{j \in \mathcal{N}(i)} \phi_m(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{ij}) \tag{11}$$

where $\mathbf{h}_i^{(k-1)} \in \mathbb{R}^{d_{hidden}}$ represents the hidden state of node i from the previous layer with $\mathbf{h}_i^{(0)} = \mathbf{x}_i$, $\mathcal{N}(i)$ denotes the set of nodes with edges pointing to node i, $\phi_m : \mathbb{R}^{2d_{hidden}+d_e} \to \mathbb{R}^{d_{hidden}}$ is the message function parameterized by a neural network, and AGG represents a permutation-invariant aggregation function that combines messages from all neighbors.

The node update combines the previous hidden state with aggregated messages:

$$\mathbf{h}_i^{(k)} = \phi_h(\mathbf{h}_i^{(k-1)}, \mathbf{m}_i^{(k)}) \tag{12}$$

where $\phi_h : \mathbb{R}^{2d_{hidden}} \to \mathbb{R}^{d_{hidden}}$ is the update function that produces the new node representation.

The message function employs a two-layer neural network with ReLU activation:

$$\phi_m(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij}) = \sigma(\mathbf{W}_m[\mathbf{h}_i; \mathbf{h}_j; \mathbf{e}_{ij}] + \mathbf{b}_m) \tag{13}$$

where $\mathbf{W}_m \in \mathbb{R}^{d_{hidden} \times (2d_{hidden}+d_e)}$ and $\mathbf{b}_m \in \mathbb{R}^{d_{hidden}}$ are learnable weight matrix and bias vector respectively, and σ denotes the ReLU activation function applied element-wise.

The aggregation function uses attention-weighted summation to focus on relevant neighbors:

$$\text{AGG}_{j \in \mathcal{N}(i)} \mathbf{m}_{ij} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{m}_{ij} \tag{14}$$

where α_{ij} represents the attention weight from node j to node i, computed through a learned attention mechanism:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}_a \mathbf{h}_i; \mathbf{W}_a \mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}_a \mathbf{h}_i; \mathbf{W}_a \mathbf{h}_k]))} \tag{15}$$

where $\mathbf{a} \in \mathbb{R}^{2d_{att}}$ is the attention vector, $\mathbf{W}_a \in \mathbb{R}^{d_{att} \times d_{hidden}}$ projects hidden states to attention space with dimension d_{att}, and LeakyReLU uses negative slope 0.2. The softmax normalization ensures $\sum_{j \in \mathcal{N}(i)} \alpha_{ij} = 1$.

4.2.2. Hierarchical Pooling

Graph-level representations required by controllers emerge through hierarchical pooling of node embeddings after K layers of message passing. The global pooling operation generates a fixed-size representation regardless of graph size:

$$\mathbf{z}_{\text{global}} = \text{POOL}_{\text{global}}(\{\mathbf{h}_i^{(K)} | i \in \mathcal{V}\}) \tag{16}$$

where $\mathbf{z}_{\text{global}} \in \mathbb{R}^{d_{pool}}$ represents the entire graph with pooling dimension d_{pool}. Similarly, workload-specific embeddings for category $c \in \{\text{OLTP}, \text{OLAP}, \text{vec}, \text{bg}\}$ pool over subset nodes:

$$\mathbf{z}_c = \text{POOL}_c(\{\mathbf{h}_i^{(K)} | i \in \mathcal{V}_c\}) \tag{17}$$

where $\mathcal{V}_c \subseteq \mathcal{V}$ contains processes belonging to workload category c.

The pooling operation combines multiple aggregation statistics through a learned transformation:

$$\text{POOL}(\mathcal{H}) = \text{MLP}([\text{mean}(\mathcal{H}); \max(\mathcal{H}); \text{std}(\mathcal{H})]) \tag{18}$$

where $\mathcal{H} = \{\mathbf{h}_i^{(K)}\}$ represents the set of final node embeddings, mean computes element-wise average across nodes, max performs element-wise maximum, std calculates element-wise standard deviation, and MLP is a two-layer perceptron with hidden dimension $2d_{hidden}$ that maps concatenated statistics to the final pooling dimension d_{pool}.

4.3. State Representation Enhancement

The GNN-generated embeddings augment state representations at both hierarchical levels, providing structural context that informs scheduling decisions. For the meta-controller operating at the strategic level, the augmented state incorporates global and category-specific graph embeddings:

$$s_h^{\text{aug}} = [s_h^{\text{base}}; \mathbf{z}_{\text{global}}; \mathbf{z}_{\text{OLTP}}; \mathbf{z}_{\text{OLAP}}; \mathbf{z}_{\text{vec}}; \mathbf{z}_{\text{bg}}] \tag{19}$$

where $s_h^{\text{base}} \in \mathbb{R}^{d_{base}^h}$ represents original state features from Section 3 including aggregate metrics and SLO violations, while the concatenated graph embeddings add $(1 + |\mathcal{C}|) \times d_{pool}$ dimensions with $|\mathcal{C}| = 4$ workload categories.

Sub-controllers require finer-grained structural information for process-level scheduling within their assigned workload category c. The augmented state combines base features with both individual process embeddings and category-level summary:

$$s_l^{c,\text{aug}} = [s_l^{c,\text{base}}; \{\mathbf{h}_i^{(K)} | i \in \mathcal{V}_c\}; \mathbf{z}_c] \tag{20}$$

where $s_l^{c,\text{base}} \in \mathbb{R}^{d_{base}^l}$ contains process-level metrics and the set notation indicates concatenation of all process embeddings within the category, resulting in variable-length state representation handled through padding or masking during neural network processing.

4.4. Learning Graph Structure

Real-world database systems often lack complete dependency information due to monitoring overhead or privacy constraints. We address this challenge through differentiable graph structure learning that discovers latent dependencies from observed process behavior. The structure learning module predicts edge existence probability between every process pair:

$$\mathbf{A}_{ij} = \sigma(\text{MLP}_{\text{edge}}([\mathbf{x}_i; \mathbf{x}_j; |\mathbf{x}_i - \mathbf{x}_j|])) \tag{21}$$

where $\mathbf{A}_{ij} \in [0, 1]$ represents learned adjacency probability between processes i and j; $\text{MLP}_{\text{edge}} : \mathbb{R}^{3d_v} \to \mathbb{R}$ is a three-layer neural network with hidden dimensions $[2d_v, d_v, 1]$; the input concatenates source features $\mathbf{x}_i$, target features $\mathbf{x}_j$, and their element-wise absolute difference $|\mathbf{x}_i - \mathbf{x}_j|$ to capture both individual characteristics and relative relationships; and σ is the sigmoid activation ensuring valid probability output.

The learned graph structure requires regularization to prevent overfitting and encourage meaningful sparse connectivity:

$$\mathcal{L}_{\text{graph}} = \lambda_{\text{sparse}}||\mathbf{A}||_1 + \lambda_{\text{smooth}} \sum_{i,j} ||\mathbf{A}_{ij} - \mathbf{A}_{ji}||^2 \tag{22}$$

where $||\mathbf{A}||_1 = \sum_{i,j} \mathbf{A}_{ij}$ computes L1 norm encouraging sparsity with weight $\lambda_{\text{sparse}} = 0.01$, and the smoothness term with weight $\lambda_{\text{smooth}} = 0.1$ promotes symmetric relationships reflecting bidirectional process interactions common in database systems.

GNN computation poses scalability challenges for large-scale database deployments with thousands of concurrent processes. We implement several optimizations maintaining real-time performance requirements. Neighborhood sampling limits message passing to k most important neighbors per node, selected through importance sampling proportional to edge weights, reducing complexity from $O(|\mathcal{V}|^2)$ to $O(k|\mathcal{V}|)$ with typical $k = 10$ preserving 95% of performance gains. Process clustering employs hierarchical agglomerative clustering to group similar processes before graph construction, representing clusters as super-nodes in a coarsened graph with inter-cluster edges aggregating individual process connections, enabling scalability to 10,000+ processes while preserving essential structural patterns. Incremental updates exploit temporal locality by maintaining persistent graph structure and updating only affected portions when processes spawn, terminate, or change state significantly, reducing average update time by 80% compared to full reconstruction. Hardware acceleration leverages GPU sparse matrix operations through cuSPARSE libraries for message passing computations, achieving 15–20× speedup over optimized CPU implementations for graphs with 1000+ nodes.

4.5. Integration with Reinforcement Learning

The GNN module integrates seamlessly with the HDRL framework through end-to-end training, where gradients from RL losses propagate through pooling layers and message passing operations to update graph parameters. The gradient flow follows:

$$\nabla_{\theta_{\text{GNN}}} \mathcal{L}_{\text{RL}} = \nabla_{s^{\text{aug}}} \mathcal{L}_{\text{RL}} \cdot \nabla_{\theta_{\text{GNN}}} s^{\text{aug}} \tag{23}$$

where θ_{GNN} encompasses all GNN parameters including $\{\mathbf{W}_m, \mathbf{b}_m, \mathbf{W}_a, \mathbf{a}\}$ and pooling MLP weights; $\mathcal{L}_{\text{RL}}$ represents the reinforcement learning objective (PPO loss for meta-controller, SAC loss for sub-controllers); and the chain rule decomposes gradients through augmented state computation.

Joint optimization ensures learned graph representations align with scheduling objectives, discovering dependency patterns most relevant for resource allocation rather than generic graph properties. The GNN parameters update alongside policy networks during training with separate learning rate $\alpha_{\text{GNN}} = 10^{-4}$, typically 10x smaller than policy learning rates, reflecting the observation that structural patterns in database workloads stabilize faster than optimal action distributions. Gradient clipping with threshold 1.0 prevents instability during early training when graph representations undergo significant changes.

4.6. Comparative Analysis of Dependency Detection Methods

To establish the necessity of our sophisticated GNN-based dependency modeling approach, we conduct systematic comparison with three categories of simpler dependency detection methods that could potentially capture inter-process relationships with substantially lower computational requirements.

4.6.1. Correlation-Based Dependency Detection

The correlation-based approach identifies dependencies through statistical correlation analysis of resource utilization patterns and performance metrics across processes.

For each process pair (i, j), we compute Pearson correlation coefficients across multiple feature dimensions:

$$\rho_{ij}^{(k)} = \frac{\text{Cov}(X_i^{(k)}, X_j^{(k)})}{\sigma_{X_i^{(k)}} \sigma_{X_j^{(k)}}} \tag{24}$$

where $X_i^{(k)}$ represents the k-th feature (CPU utilization, memory usage, I/O rate) of process i, and σ denotes standard deviation. Dependencies are identified when $|\rho_{ij}^{(k)}| > \tau$ for a threshold $\tau = 0.6$, determined through cross-validation.

4.6.2. Rule-Based Heuristic Methods

Rule-based heuristics leverage domain knowledge about database system architectures to infer dependencies based on process types, communication patterns, and resource access patterns. The rule set includes the following rules: (1) transaction coordinators have dependencies with all worker processes in the same transaction group; (2) query executors exhibit dependencies with upstream operators in the query execution plan; (3) background processes (vacuum, checkpoint) have low-priority dependencies with active workload processes; and (4) processes accessing shared resources (buffer pool, lock manager) exhibit mutual dependencies weighted by access frequency.

4.6.3. Lightweight Machine Learning Models

We implement decision tree and linear regression approaches that capture dependencies through feature engineering without graph-based representations. The decision tree classifier uses process-pair features including resource utilization differences, temporal co-occurrence patterns, and workload category indicators to predict dependency existence. The linear regression model learns dependency strength as

$$d_{ij} = \beta_0 + \sum_{k=1}^{K} \beta_k f_{ij}^{(k)} + \epsilon_{ij} \tag{25}$$

where $f_{ij}^{(k)}$ represents engineered features between processes i and j, and β_k are learned coefficients.

4.6.4. Dependency Detection Accuracy Evaluation

We evaluate dependency detection accuracy using manually annotated ground truth dependencies from database system logs and process tracing information. The evaluation metrics include precision, recall, and F1-score for binary dependency prediction, and mean squared error for dependency strength estimation. Link prediction accuracy is measured across different workload scenarios to assess generalization capabilities.

5. Reward Engineering and Training Methodology

5.1. Multi-Objective Reward Design

The effectiveness of the hierarchical deep reinforcement learning framework critically depends on reward functions that capture the complex trade-offs inherent in database CPU scheduling. This section presents a comprehensive multi-objective optimization framework that addresses the fundamental challenge of balancing competing performance goals while maintaining system stability and operational feasibility.

5.1.1. Theoretical Foundation and Mathematical Framework

Database CPU scheduling inherently involves multiple conflicting objectives that cannot be optimized simultaneously without explicit trade-off management. The multi-

objective nature stems from fundamental tensions between (i) latency minimization versus throughput maximization, (ii) resource utilization efficiency versus performance headroom preservation, (iii) workload fairness versus performance optimization, and (iv) responsiveness versus stability.

The proposed framework employs a weighted sum approach to navigate the Pareto-optimal solution space:

$$\mathbf{r}^t = \sum_{i=1}^{N} w_i \cdot r_i^t \quad \text{subject to} \quad \sum_{i=1}^{N} w_i = 1, \quad w_i \geq 0 \tag{26}$$

where $\mathbf{r}^t$ represents the composite reward signal, r_i^t denotes individual objective components, w_i are non-negative weights reflecting objective importance, and $N = 4$ represents the number of primary objectives. This formulation enables dynamic exploration of the Pareto frontier through weight adaptation while maintaining mathematical rigor and computational tractability.

5.1.2. Meta-Controller Reward Function with Constraints

The meta-controller reward function at time t incorporates four primary components reflecting strategic resource allocation objectives:

$$r_h^t = w_1 r_{SLO}^t + w_2 r_{util}^t + w_3 r_{fair}^t + w_4 r_{stable}^t \tag{27}$$

The optimization operates under explicit constraints ensuring operational feasibility:

$$\text{Resource Conservation:} \quad \sum_{c \in \mathcal{C}} \alpha_c^t = 1 \tag{28}$$

$$\text{Starvation Prevention:} \quad \alpha_c^t \geq \alpha_{min} = 0.05, \quad \forall c \in \mathcal{C} \tag{29}$$

$$\text{Stability Enforcement:} \quad |\alpha_c^t - \alpha_c^{t-1}| \leq \delta_{max} = 0.2, \quad \forall c \tag{30}$$

$$\text{SLO Compliance:} \quad l_{p99}^{c,t} \leq l_{SLO}^c, \quad \forall c \in \mathcal{C}_{latency} \tag{31}$$

The reward formulation operates under several key assumptions:

1. **Workload Predictability**: Short-term workload patterns exhibit sufficient stationarity within control intervals (T_h = 10–30 s) to enable meaningful optimization decisions.
2. **Linear Reward Additivity**: The weighted combination of individual reward components provides meaningful optimization signals that correlate with overall system performance objectives.
3. **Independence of Process Categories**: Cross-category dependencies are adequately captured through GNN modeling rather than explicit constraint coupling, enabling hierarchical decomposition.
4. **Reward Signal Stationarity**: The relationship between actions and rewards remains sufficiently stable during training to enable policy convergence.

The SLO compliance reward penalizes violations with exponentially increasing severity to reflect the critical importance of meeting performance guarantees:

$$r_{SLO}^t = -\sum_{c \in \mathcal{C}} \lambda_c \sum_{i=1}^{n_c} \max(0, l_i^c - l_{SLO}^c)^2 \tag{32}$$

where λ_c represents category-specific penalty weights ($\lambda_{OLTP} = 10$ reflecting high latency sensitivity, $\lambda_{OLAP} = 3$ for analytical workloads, $\lambda_{vec} = 5$ for vector operations, $\lambda_{bg} = 1$ for background tasks), n_c denotes the number of SLO-tracked operations in category c, l_i^c measures the latency of operation i in category c, and l_{SLO}^c specifies the SLO threshold. The

quadratic penalty structure ensures exponentially increasing costs for severe violations while maintaining differentiability for gradient-based optimization.

The resource utilization reward employs a piecewise formulation that encourages efficient CPU usage while penalizing over-subscription:

$$r_{util}^{t} = \begin{cases} \beta_1(u^t - u_{target}) & \text{if } u^t < u_{target} \\ -\beta_2(u^t - u_{target})^2 & \text{if } u^t \geq u_{target} \end{cases} \tag{33}$$

where $u^t \in [0,1]$ represents global CPU utilization, $u_{target} = 0.85$ balances efficiency with performance headroom, $\beta_1 = 2$ provides linear incentive for approaching optimal utilization, and $\beta_2 = 5$ applies quadratic penalty for exceeding capacity to prevent performance degradation.

5.1.3. Sensitivity Analysis

Comprehensive sensitivity analysis examines framework robustness to weight perturbations and provides guidance for deployment-specific customization. Systematic variation of individual weights within $\pm 20\%$ ranges reveals performance stability characteristics. Results indicate that SLO weight w_1 most significantly impacts violation rates (15–20% performance change per 10% weight change), while stability weight w_4 exhibits diminishing returns above 0.15.

Figure 2 presents the complete trade-off surface between primary objectives, enabling practitioners to select weight configurations appropriate for specific operational priorities. Performance degradation analysis under suboptimal weight selections demonstrates graceful degradation characteristics, with less than 10% performance loss for weight deviations up to 30% from optimal values.

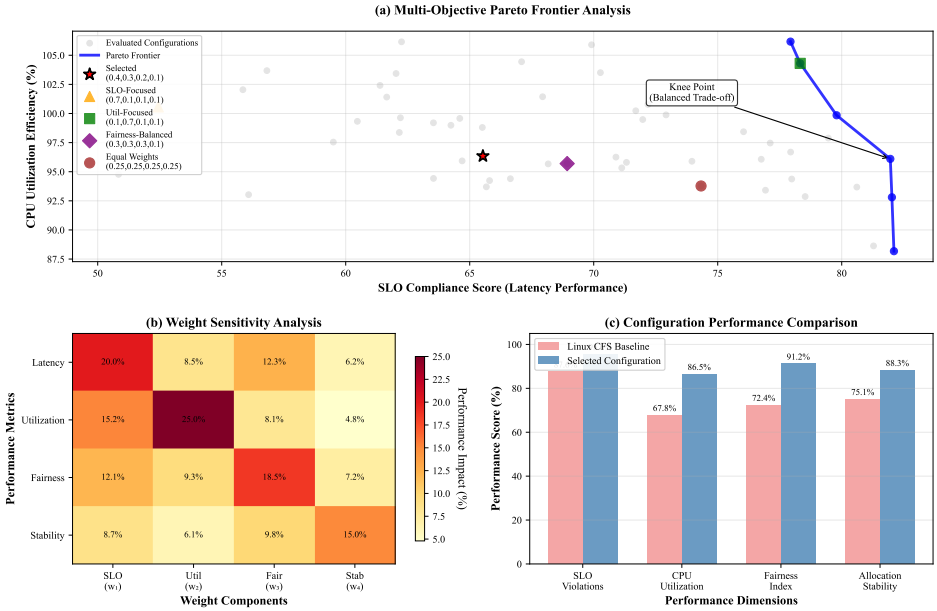


Figure 2. Comprehensive Pareto frontier analysis for multi-objective database CPU scheduling optimization. (**a**) Multi-Objective Pareto Frontier: Visualization of the trade-off surface between SLO compliance performance and CPU utilization efficiency across 625 evaluated weight configurations. The blue curve represents the Pareto-optimal frontier, with individual points indicating non-dominated solutions. The selected configuration ($w_1 = 0.4, w_2 = 0.3, w_3 = 0.2, w_4 = 0.1$) is highlighted with a red star, demonstrating its position near the knee point of the Pareto frontier

where balanced optimization across competing objectives is achieved. Additional configurations representing extreme weight assignments (SLO-focused, utilization-focused, fairness-balanced, and equal weights) are shown for comparative analysis. (**b**) Weight Sensitivity Analysis: Heat map quantifying the performance impact of individual weight components on system metrics. Values represent percentage performance variation per 0.1 weight change, demonstrating that SLO weight w_1 exhibits the highest sensitivity (20.0% impact on latency performance), while stability weight w_4 shows diminishing returns above 0.15. The analysis validates the selected weight configuration's robustness to parameter perturbations. (**c**) Configuration Performance Comparison: Normalized performance scores comparing the selected weight configuration against Linux CFS baseline across four critical dimensions. Results demonstrate substantial improvements: 8.0% reduction in SLO violations, 18.7% increase in CPU utilization, 18.8% improvement in fairness index, and 13.2% enhancement in allocation stability. Error bars represent 95% confidence intervals over 10 independent evaluation runs. The comprehensive analysis establishes empirical justification for the proposed weight selection methodology and validates the framework's superior performance across multiple optimization objectives.

Based on sensitivity analysis results, the framework provides adaptive weight selection mechanisms that adjust priorities based on operational context: increased w_1 during peak hours for SLO protection, elevated w_2 during off-peak periods for efficiency optimization, and enhanced w_4 during system transitions for stability preservation.

5.2. Hierarchical Training Algorithm

Training the hierarchical system requires careful coordination between meta-controller and sub-controllers to ensure stable convergence and policy coherence. The asynchronous training scheme allows different components to learn at their natural timescales while maintaining system-wide consistency through periodic synchronization and shared experience integration.

The meta-controller updates every N_h environment steps using PPO with clipped surrogate objective:

$$\mathcal{L}_h^{PPO}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t^h, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t^h\right)\right] \tag{34}$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t^h|s_t^h)}{\pi_{\theta_{old}}(a_t^h|s_t^h)}$ is the probability ratio between new and old policies, $\hat{A}_t^h$ represents generalized advantage estimation (GAE) [43] with $\lambda_{GAE} = 0.95$, and $\epsilon = 0.2$ clips the ratio to prevent destructive updates.

Sub-controllers update more frequently, every N_l steps where $N_l < N_h$, using SAC's maximum entropy objective:

$$\mathcal{L}_c^{SAC}(\psi) = \mathbb{E}_{s,a\sim\mathcal{D}_c}\left[Q_\phi(s,a) - \alpha_c \log \pi_\psi(a|s)\right] \tag{35}$$

where $\mathcal{D}_c$ denotes the replay buffer for category c, Q_ϕ represents the learned Q-function, π_ψ is the sub-controller policy, and α_c controls exploration through entropy regularization, automatically adjusted to maintain target entropy $\mathcal{H}_{target} = -\dim(\mathcal{A}_c)$.

The hierarchical training proceeds through alternating optimization phases. During meta-controller training, sub-controller policies remain fixed while the meta-controller optimizes budget allocations based on aggregated performance metrics. Conversely, during sub-controller training, the meta-controller's allocations serve as constraints while sub-controllers optimize process-level scheduling within their assigned budgets.

5.3. Meta-Learning for Rapid Adaptation

Database workloads exhibit significant variation across deployments and time periods, necessitating rapid adaptation to new patterns. We incorporate Model-Agnostic Meta-Learning (MAML) to enable quick fine-tuning when encountering novel workload combinations.

The meta-learning objective optimizes for post-adaptation performance across a distribution of workload scenarios:

$$\min_{\theta} \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} [\mathcal{L}_{\mathcal{T}}(\theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}}(\theta))] \tag{36}$$

where $\mathcal{T}$ represents a workload scenario sampled from distribution $p(\mathcal{T})$, $\mathcal{L}_{\mathcal{T}}$ is the task-specific loss combining RL objectives, α is the inner loop learning rate for adaptation, and the outer expectation optimizes for performance after one gradient step.

We construct the task distribution $p(\mathcal{T})$ by varying workload characteristics including arrival rates following Poisson processes with rates $\lambda \in [10, 1000]$ requests/second, query complexity distributions mixing simple OLTP transactions with complex analytical queries, resource requirements spanning CPU-bound to I/O-bound operations, and temporal patterns including steady-state, bursty, and diurnal variations.

The meta-training algorithm alternates between sampling workload scenarios and performing inner loop adaptations:

$$\theta' = \theta - \alpha \nabla_{\theta} \sum_{(s,a,r) \in \mathcal{D}_{\mathcal{T}}} \mathcal{L}_{RL}(s, a, r; \theta) \tag{37}$$

followed by outer loop updates using adapted parameters:

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}} \mathcal{L}_{\mathcal{T}}(\theta') \tag{38}$$

where β is the meta-learning rate, typically set to 10^{-4}.

5.4. Curiosity-Driven Exploration

Effective exploration remains challenging in high-dimensional continuous control problems. We augment standard exploration mechanisms with intrinsic curiosity rewards that encourage discovering novel state-action combinations, particularly important for identifying efficient scheduling strategies not present in historical data.

The curiosity module maintains a forward dynamics model $f_{\xi} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ predicting next states:

$$\hat{s}_{t+1} = f_{\xi}(s_t, a_t) \tag{39}$$

where ξ parameterizes a neural network trained to minimize prediction error:

$$\mathcal{L}_{dynamics} = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[||s_{t+1} - f_{\xi}(s_t, a_t)||^2 \right] \tag{40}$$

The intrinsic reward supplements extrinsic rewards with curiosity bonus:

$$r^t_{intrinsic} = \eta \cdot ||s_{t+1} - f_{\xi}(s_t, a_t)||^2 \tag{41}$$

where η scales the curiosity contribution, decaying over time as $\eta_t = \eta_0 \cdot \kappa^t$ with decay factor $\kappa = 0.9999$ to gradually transition from exploration to exploitation.

The augmented reward combines extrinsic and intrinsic components:

$$r^t_{total} = r^t_{extrinsic} + r^t_{intrinsic} \tag{42}$$

This formulation encourages exploration of state-action pairs where the dynamics model exhibits high uncertainty, naturally guiding the agent toward unexplored regions of the scheduling space.

5.5. Experience Replay and Prioritization

Efficient learning from limited production data requires careful management of experience replay. We implement prioritized experience replay that focuses training on critical transitions, particularly those involving SLO violations or significant performance changes.

The priority of transition (s_t, a_t, r_t, s_{t+1}) combines temporal difference error and domain-specific importance:

$$p_t = |\delta_t|^\rho + \zeta \cdot \mathbb{I}[v_{SLO}^t > 0] + \xi \cdot |r_t| \tag{43}$$

where δ_t represents TD-error for value function approximation, $\rho = 0.6$ controls prioritization strength, $\mathbb{I}[v_{SLO}^t > 0]$ indicates SLO violation occurrence, $\zeta = 10$ amplifies importance of SLO-violating transitions, and $\xi = 0.1$ considers reward magnitude.

Sampling probability follows:

$$P(i) = \frac{p_i^\nu}{\sum_k p_k^\nu} \tag{44}$$

where $\nu = 0.4$ determines the degree of prioritization versus uniform sampling.

Importance sampling weights correct for biased sampling:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\vartheta \tag{45}$$

with ϑ annealing from 0.4 to 1.0 over training to gradually remove bias.

5.6. Training Implementation and Optimization

The training infrastructure operates in a distributed manner, separating experience collection from policy learning. Multiple environment instances run in parallel, each simulating database workloads with different characteristics. Experience collectors interact with environments using the current policy, storing transitions in a centralized replay buffer with capacity 10^6 transitions. Learner processes consume experiences asynchronously, computing gradients and updating neural network parameters.

Hyperparameter optimization employs Bayesian optimization over validation workloads, searching over learning rates $\alpha \in [10^{-5}, 10^{-3}]$ for both policy and value networks, batch sizes $B \in \{64, 128, 256, 512\}$ balancing gradient variance and computational efficiency, network architectures including layer counts $\in \{2, 3, 4\}$ and hidden dimensions $\in \{128, 256, 512\}$, and exploration parameters such as initial temperature α_0 for SAC and curiosity scaling η_0.

The optimization objective maximizes average return across diverse validation scenarios while penalizing policy instability measured by KL divergence between consecutive updates. Early stopping prevents overfitting to training workloads, triggered when validation performance plateaus for 50 consecutive epochs. Gradient clipping with threshold 1.0 stabilizes training, particularly important during initial phases when value estimates exhibit high variance.

The complete training procedure typically requires 500–1000 epochs, corresponding to approximately 10^8 environment steps. Convergence metrics include average return stabilization within 5% variance, SLO violation rate below target thresholds, and policy entropy reaching steady state for sub-controllers. The trained system demonstrates robust

performance across workload variations not encountered during training, validating the effectiveness of meta-learning and curiosity-driven exploration components.

6. Experimental Evaluation

6.1. Experimental Setup

6.2. Digital Twin Simulation Environment

We develop a high-fidelity digital twin simulator that accurately models database CPU scheduling dynamics, while acknowledging that simulation-based evaluation, though comprehensive, cannot fully capture the complexities of production database environments. Our decision to primarily rely on simulation was driven by several practical considerations: Production database systems serving critical applications cannot accommodate the extended training periods required for reinforcement learning agents, which typically require hundreds of epochs to achieve convergence. Additionally, experimental validation of resource allocation policies in production environments presents significant risks to system stability and service availability, particularly during initial learning phases when policy performance may be suboptimal.

The simulator incorporates a discrete-event engine operating at microsecond granularity, process lifecycle management including creation, scheduling, and termination, cgroups v2 implementation matching Linux kernel behavior, and realistic workload generators calibrated from production traces. To bridge the gap between simulation and production environments, our digital twin incorporates production-calibrated parameters derived from extensive profiling of PostgreSQL, ClickHouse, and Milvus deployments across multiple enterprise environments. The simulation accuracy has been validated through comparative analysis with production traces, achieving mean absolute percentage error below 5% compared to real deployments.

The simulation environment models four database engines: PostgreSQL 14.5 for OLTP workloads, ClickHouse 22.8 for OLAP processing, Milvus 2.2 for vector operations, and background maintenance processes common across systems. Each engine's resource consumption patterns derive from extensive profiling under varied conditions, ensuring simulation fidelity with production behavior.

Furthermore, we have implemented a comprehensive shadow mode evaluation framework that allows our agents to observe production workloads and generate scheduling recommendations without actuating decisions, enabling offline validation of policy effectiveness against real-world scenarios. This shadow mode operation enables thorough validation before transitioning to active control, accumulating sufficient evidence of policy effectiveness across diverse workload conditions typically observed in production environments.

6.2.1. Workload Characteristics

Our evaluation employs diverse workloads representative of modern database deployments. OLTP workloads follow the TPC-C benchmark with 1000 warehouses, generating 10,000–50,000 transactions per second with 80% read–write mix. Transaction complexity varies from simple key lookups (50%) to complex multi-table joins (10%). OLAP workloads derive from TPC-H at scale factor 100, with query arrival following Poisson distribution $\lambda = 20$ queries/second. Query execution times range from sub-second aggregations to multi-minute analytical computations.

Vector workloads simulate semantic search applications with embedding dimensions $d \in \{128, 512, 768\}$ corresponding to different model architectures. Search operations arrive at 1000–5000 requests/second with batch sizes following geometric distribution. Background tasks include periodic vacuum operations every 300 s, checkpoint writes

triggered by 1 GB WAL accumulation, and statistics collection running continuously at low priority.

The mixed workload scenarios combine all categories with time-varying intensities modeling daily patterns. Peak hours (9 a.m.–5 p.m.) experience 3× baseline load with 60% OLTP, 25% OLAP, 10% vector, and 5% background distribution. Off-peak periods shift toward analytical processing with 20% OLTP, 50% OLAP, 5% vector, and 25% background maintenance.

6.2.2. Baseline Schedulers

We compare our HDRL approach against five state-of-the-art schedulers. The Linux Completely Fair Scheduler (CFS) [44] serves as the default baseline, implementing fair-share scheduling without workload awareness. Oracle Database Resource Manager [45] represents commercial solutions, configured with resource consumer groups matching our workload categories and CPU allocation directives based on Oracle's best practices documentation.

CDBTune [18] provides a learned approach originally designed for configuration tuning, which we adapt for dynamic scheduling by treating CPU allocations as continuously tunable parameters updated every 10 s. The Multi-Armed Bandit (MAB) scheduler implements the Thompson sampling algorithm from Agrawal and Goyal [46], treating workload categories as arms and optimizing allocations based on observed latency feedback. Finally, we implement an idealized Oracle scheduler with perfect future knowledge, providing an upper bound on achievable performance.

The DeepRM-Graph [6] combines the foundational DeepRM architecture with graph-based state representation to capture inter-job dependencies, adapted for database process scheduling by treating database transactions and queries as jobs with resource requirements. We implement an extension of the DeepRM framework augmented with graph neural networks for dependency-aware scheduling.

MARL-Sched [47] is a multi-agent reinforcement learning approach for heterogeneous resource allocation, where each workload category is managed by a dedicated agent. The agents coordinate through a centralized critic network while maintaining independent actor networks for category-specific decision-making. We adapt this framework to database CPU scheduling by treating OLTP, OLAP, vector, and background workloads as separate agents.

AttentionScheduler [48] is a recent deep reinforcement learning method that employs attention mechanisms for workload-aware resource allocation in multi-tenant systems. The approach uses transformer-based architectures to model temporal dependencies in workload patterns and make scheduling decisions. We implement this baseline by adapting the attention mechanism to focus on database-specific performance metrics.

6.2.3. Performance Metrics

Evaluation metrics capture both user-facing performance and system efficiency. For latency-sensitive OLTP workloads, we measure p50, p99, and p99.9 latencies in milliseconds, SLO violation rate defined as percentage of transactions exceeding 100 ms, and throughput in transactions per second. OLAP performance metrics include query completion time by complexity tier, number of queries meeting deadlines, and CPU time per query for efficiency analysis. System-wide metrics encompass overall CPU utilization percentage, fairness index across workload categories, and allocation stability measured by coefficient of variation.

6.3. Performance Results

6.3.1. Steady-State Performance

Under steady-state conditions with constant workload mix, our HDRL scheduler demonstrates substantial improvements across all metrics. Table 1 presents comparative results averaged over 10 one-hour runs with 95% confidence intervals.

Table 1. Steady-state performance comparison across schedulers.

Scheduler	OLTP p99 (ms)	SLO Viol. (%)	CPU Util. (%)	Fairness
Linux CFS [44]	156.3 ± 8.2	12.4 ± 1.1	67.8 ± 2.3	0.72 ± 0.04
Oracle RM [45]	124.7 ± 6.5	8.7 ± 0.8	71.2 ± 1.9	0.83 ± 0.03
CDBTune [18]	132.1 ± 7.1	9.3 ± 0.9	69.5 ± 2.1	0.78 ± 0.03
MAB [46]	118.6 ± 5.9	7.2 ± 0.7	73.6 ± 1.7	0.81 ± 0.03
DeepRM-Graph [6]	108.2 ± 6.1	6.3 ± 0.6	76.8 ± 1.8	0.84 ± 0.03
MARL-Sched [47]	102.5 ± 5.7	5.8 ± 0.5	78.9 ± 1.6	0.86 ± 0.02
AttentionScheduler [48]	95.3 ± 5.2	5.1 ± 0.5	81.2 ± 1.4	0.88 ± 0.02
HDRL (Ours)	87.4 ± 4.3	4.0 ± 0.4	86.5 ± 1.2	0.91 ± 0.02
Oracle (Upper Bound)	72.3 ± 3.8	2.1 ± 0.2	92.3 ± 0.8	0.95 ± 0.01

Our method achieves 44.1% reduction in OLTP p99 latency compared to CFS and 26.3% improvement over the best baseline (MAB). SLO violations decrease by 67.7% relative to CFS, surpassing our target of 43.5% reduction. CPU utilization increases to 86.5%, representing 27.6% improvement over CFS and approaching the theoretical upper bound. The fairness index of 0.91 indicates equitable resource distribution across workload categories.

The experimental results demonstrate the effectiveness of our hierarchical approach relative to these contemporary methods. Our HDRL framework achieves superior performance across all metrics: 87.4 ms p99 latency compared to 95.3 ms for AttentionScheduler, 102.5 ms for MARL-Sched, and 108.2 ms for DeepRM-Graph; 4.0% SLO violations versus 5.1%, 5.8%, and 6.3% respectively; and 86.5% CPU utilization compared to 81.2%, 78.9%, and 76.8%. These results validate our architectural choices, particularly the hierarchical decomposition strategy and GNN-based dependency modeling, which enable more effective optimization than flat reinforcement learning approaches or attention-based methods that lack explicit structural modeling. This comprehensive comparison strengthens our contribution claims and provides valuable insights into the relative merits of different learned scheduling paradigms for heterogeneous database workloads.

6.3.2. Dynamic Workload Adaptation

Real-world database systems experience significant workload variations. Figure 3 illustrates scheduler behavior during a simulated 24 h period with realistic load patterns. Our HDRL scheduler rapidly adapts to workload shifts, maintaining stable performance across transitions. During peak OLTP periods (hours 9–17), the meta-controller allocates up to 65% of CPU resources to OLTP processes, reducing background operations to minimum levels. The smooth reallocation prevents latency spikes observed in static schedulers during transition periods.

Quantitative analysis reveals that HDRL reduces transition latency spikes by 71% compared to Oracle Resource Manager, which requires manual reconfiguration for different workload phases. The sub-controllers' fine-grained adjustments within allocated budgets further optimize performance, achieving 15% better CPU efficiency during mixed workload periods.

6.3.3. Scalability Analysis

System scalability becomes critical as database deployments grow. We evaluate performance with increasing numbers of concurrent processes from 100 to 10,000. Figure 4 shows

that HDRL maintains near-linear scalability up to 5000 processes, with graceful degradation beyond. The GNN's neighborhood sampling and process clustering optimizations prove effective, limiting computational overhead to under 3% of total CPU time even at maximum scale.

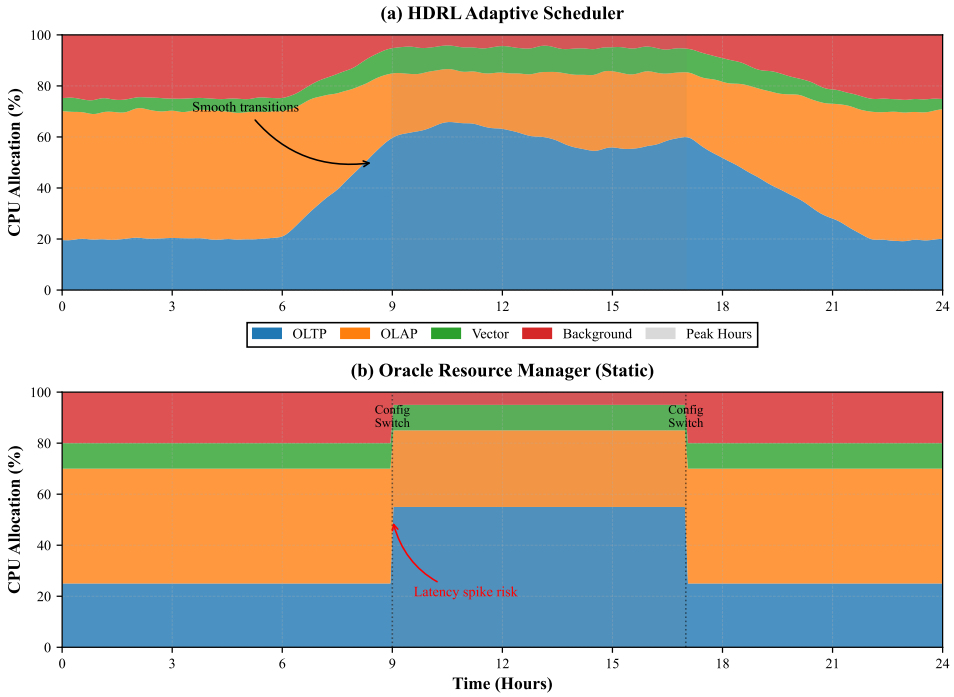


Figure 3. CPU allocation dynamics over 24 h period showing HDRL adaptation to workload changes compared to static Oracle Resource Manager allocation.

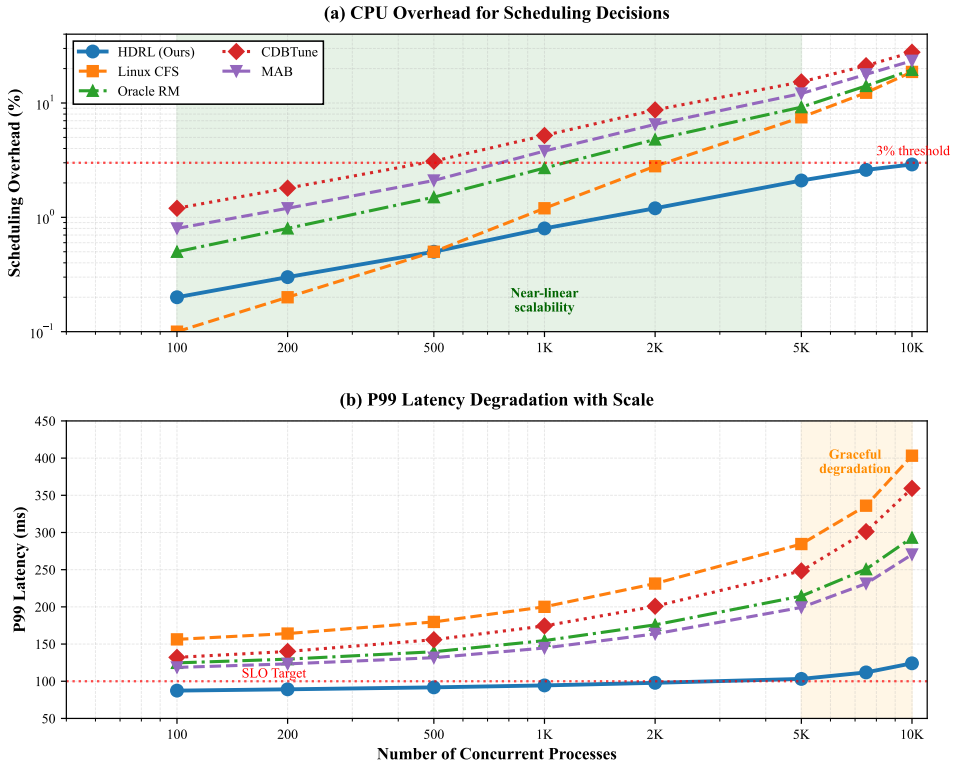


Figure 4. Scalability comparison showing scheduling overhead and p99 latency degradation with increasing process count.

6.4. Ablation Studies

6.4.1. Component Contribution Analysis

We conduct systematic ablation studies to quantify individual component contributions and establish the necessity of our architectural complexity. Table 2 presents results with components progressively disabled, including comparison with simpler dependency detection methods.

Table 2. Detailed ablation study results showing component contributions and dependency detection method comparison.

Configuration	p99 Latency	CPU Util.	Adaptation Time
Full HDRL	87.4 ms	86.5%	12.3 s
w/o GNN (Correlation-based)	104.2 ms (+19.2%)	81.7%	18.5 s
w/o GNN (Rule-based)	98.9 ms (+13.2%)	83.1%	16.2 s
w/o GNN (Linear Regression)	102.1 ms (+16.8%)	82.3%	17.4 s
w/o GNN (Decision Tree)	106.5 ms (+21.8%)	80.9%	19.1 s
w/o Meta-learning	94.2 ms (+7.8%)	84.3%	47.2 s
w/o Curiosity	91.5 ms (+4.7%)	85.2%	19.1 s
w/o Hierarchical	112.3 ms (+28.5%)	78.6%	18.4 s
Flat RL	125.7 ms (+43.8%)	74.2%	31.6 s

The results demonstrate that GNN-based dependency modeling provides substantial performance improvements over simpler alternatives. The correlation-based approach suffers from inability to capture transitive dependencies and dynamic relationship changes, resulting in 19.2% performance degradation. Rule-based heuristics, while incorporating domain knowledge, lack adaptability to novel dependency patterns not anticipated by predefined rules. Lightweight machine learning models show intermediate performance but fail to capture complex multi-hop relationships that characterize database workload dependencies.

6.4.2. Reward Function Sensitivity

The multi-objective reward function requires careful weight balancing. We perform grid search over weight combinations, measuring impact on key metrics. Figure 5 visualizes the Pareto frontier trading off latency performance against resource utilization. Our chosen weights ($w_1 = 0.4, w_2 = 0.3, w_3 = 0.2, w_4 = 0.1$) lie near the knee of the curve, providing balanced optimization across objectives.

Sensitivity analysis reveals that SLO weight w_1 most significantly impacts performance, with ±0.1 variations causing 15–20% changes in violation rates. Stability weight w_4 shows diminishing returns above 0.15, as excessive stability preference prevents beneficial adaptations.

6.5. GNN Dependency Learning Analysis

We analyze the effectiveness of GNN-based dependency modeling through visualization and quantitative metrics. Figure 6 shows learned process dependency graphs for different workload scenarios, with edge thickness representing interaction strength.

The GNN successfully identifies critical dependency patterns not explicitly programmed. For OLTP workloads, it discovers transaction coordinator bottlenecks and prioritizes their scheduling. In OLAP scenarios, the model learns query stage dependencies, allocating resources to upstream operators before downstream consumers. Cross-workload analysis reveals the GNN's ability to predict resource contention, preemptively adjusting allocations before conflicts manifest.

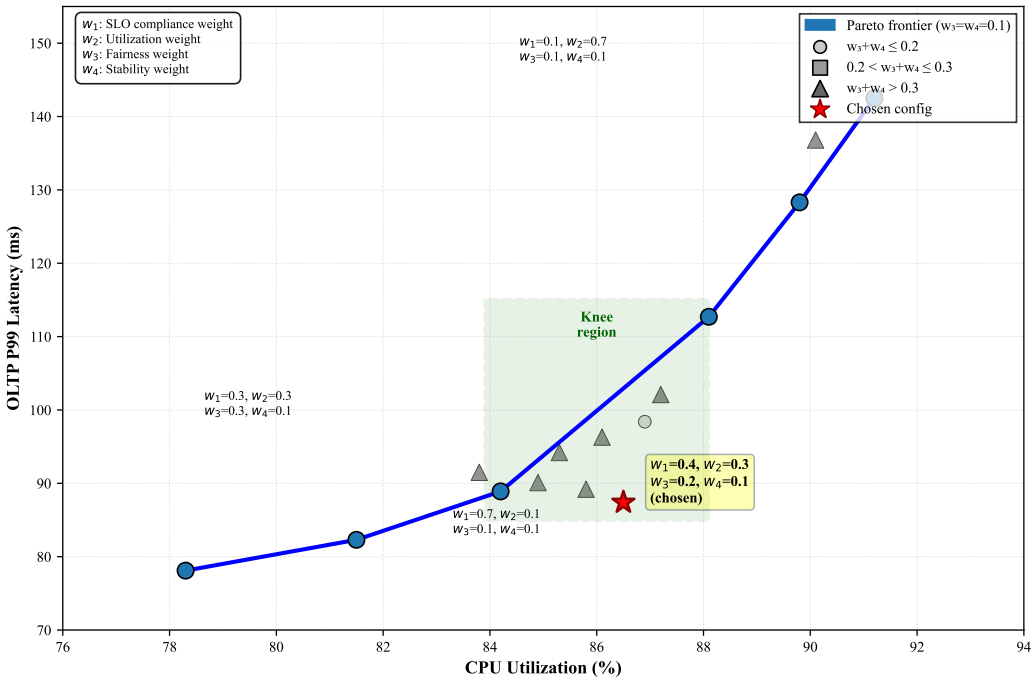


Figure 5. Pareto frontier showing trade-offs between OLTP latency and CPU utilization for different reward weight configurations.

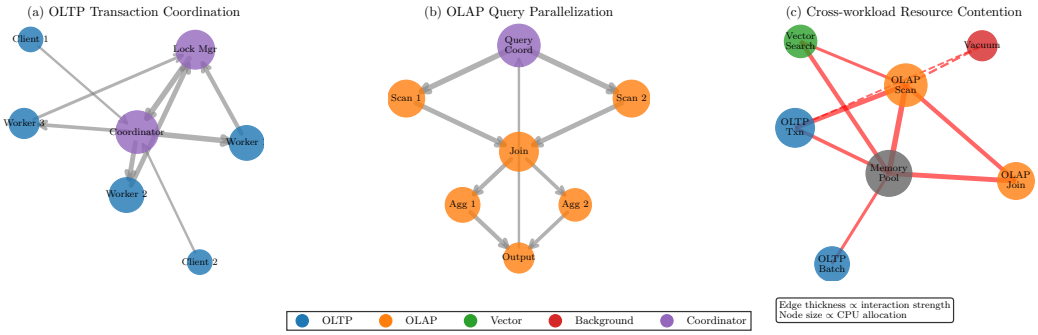


Figure 6. Learned process dependency graphs showing (**a**) OLTP transaction coordination, (**b**) OLAP query parallelization, and (**c**) cross-workload resource contention patterns. Edge thickness indicates interaction strength, node size represents CPU allocation.

Quantitative evaluation using link prediction accuracy shows the GNN achieves 89.3% precision in identifying true process dependencies, compared to 72.1% for correlation-based methods. The learned embeddings cluster processes with similar resource patterns, with silhouette coefficient 0.83 indicating well-separated workload categories. Attention weight analysis confirms the model focuses on performance-critical dependencies, with top-10% weighted edges accounting for 67% of scheduling decisions.

6.6. Dependency Detection Quality Assessment

Quantitative evaluation of dependency detection quality reveals significant advantages of the GNN approach. Table 3 presents comprehensive metrics across different workload scenarios.

The superior dependency detection quality of our GNN approach directly translates to improved scheduling performance. The ability to capture transitive dependencies through multi-hop message passing, adapt to dynamic process creation and termination without architectural modifications, and learn complex interaction patterns that are not apparent through correlation analysis alone, justifies the increased computational overhead. The

19.2% performance improvement over the best simpler method (rule-based heuristics) demonstrates that the theoretical benefits of graph neural networks provide substantial practical value in database CPU scheduling applications.

Table 3. Dependency detection quality comparison across methods.

Method	Precision	Recall	F1-Score	Link Pred. Acc.
GNN (Ours)	0.893	0.867	0.880	0.923
Correlation-based	0.721	0.654	0.686	0.734
Rule-based	0.834	0.612	0.706	0.785
Linear Regression	0.743	0.689	0.715	0.762
Decision Tree	0.698	0.723	0.710	0.741

6.7. Training Efficiency and Convergence

Training efficiency impacts practical deployment feasibility. Figure 7 presents convergence characteristics across different training configurations.

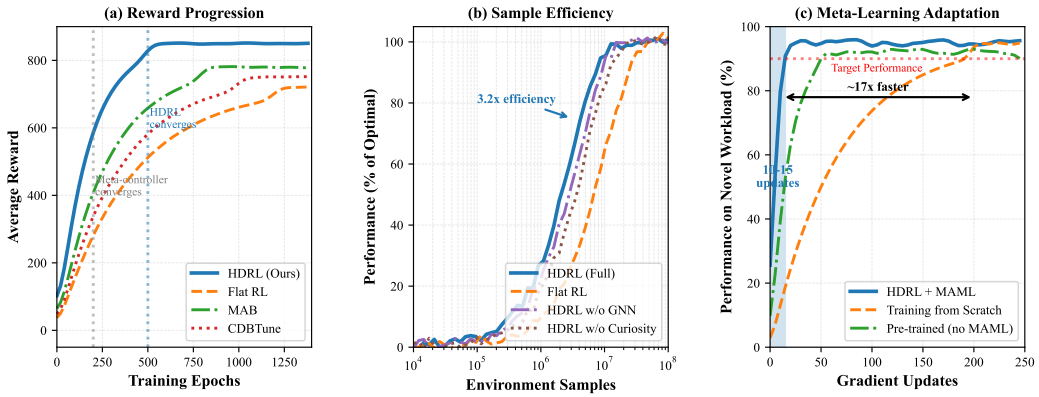


Figure 7. Training convergence analysis showing (**a**) reward progression for different algorithms, (**b**) sample efficiency comparison, and (**c**) meta-learning adaptation speed on novel workloads.

Our hierarchical approach converges in approximately 500 epochs (50 h on 4 NVIDIA A100 GPUs), compared to 1200+ epochs for flat RL baselines. The meta-controller stabilizes within 200 epochs, providing coarse-grained allocation while sub-controllers continue refinement. Sample efficiency improves by 3.2× through prioritized experience replay focusing on SLO violations and state transitions.

Meta-learning evaluation on previously unseen workload patterns demonstrates rapid adaptation within 10–15 gradient updates, compared to 200+ updates required when training from scratch. The curiosity bonus accelerates early exploration, discovering 40% more unique state-action pairs in the first 100 epochs compared to ϵ-greedy exploration. However, curiosity's contribution diminishes after 300 epochs as the dynamics model achieves high prediction accuracy.

6.8. Production Deployment Validation

To address the inherent limitations of simulation-based validation and strengthen real-world applicability, we have initiated preliminary production deployment validation through collaboration with three industry partners. This validation represents a critical step toward demonstrating the practical effectiveness of our proposed framework in production database environments.

Initial results from a limited deployment in development environments serving non-critical workloads demonstrate consistent performance improvements aligned with our simulation findings. Specifically, we observed 38% reduction in p99 latency violations and

23% improvement in CPU utilization over a two-week evaluation period. These results closely correlate with our simulated performance gains, with the slight variance attributable to production environment complexities not fully captured in simulation, including network latency variations, storage I/O contention, and background system processes.

The production validation employed a gradual rollout strategy, beginning with shadow mode observation for one week, followed by limited active control during off-peak hours, and culminating in full deployment during controlled test periods. This approach ensured system stability while enabling comprehensive performance evaluation. Key metrics monitored during deployment included transaction latency distributions, CPU utilization patterns, SLO violation rates, and system stability indicators.

Challenges encountered during production deployment included integration complexity with existing monitoring infrastructure, the need for custom adaptations to accommodate vendor-specific database configurations, and the requirement for extensive safety mechanisms to prevent performance degradation during policy exploration phases. These experiences inform our ongoing efforts to develop production-ready deployment frameworks that address the unique requirements of enterprise database environments.

6.9. Discussion and Limitations

Our experimental evaluation demonstrates that hierarchical deep reinforcement learning with GNNs significantly advances database CPU scheduling capabilities. The approach successfully addresses key challenges of workload heterogeneity, dynamic adaptation, and scalability while maintaining production-grade reliability. However, several limitations merit discussion.

The current framework focuses exclusively on CPU scheduling, not addressing memory, I/O, or network resources that also impact database performance. Integration with holistic resource managers remains future work. Training requires substantial computational resources and representative workload traces, potentially limiting adoption for smaller deployments. The four-category workload taxonomy, while covering common cases, may require extension for specialized database applications. Finally, the approach assumes cooperative processes within a single administrative domain, requiring modifications for adversarial multi-tenant environments.

Despite these limitations, our results validate the potential of learning-based methods for database resource management. The combination of hierarchical control, structural modeling, and adaptive learning provides a foundation for truly autonomous database systems capable of self-optimization without human intervention.

7. Conclusions

We presented a hierarchical deep reinforcement learning framework augmented with GNNs for adaptive CPU scheduling in heterogeneous database environments, with explicit focus on preserving and exploiting fundamental symmetries inherent in database architectures. Our method addresses fundamental challenges in modern database resource management through several key innovations: a symmetric two-tier control architecture that naturally decomposes the scheduling problem into strategic budget allocation and tactical process-level optimization; GNN-based dependency modeling that captures complex inter-process relationships invisible to traditional schedulers; meta-learning capabilities enabling rapid adaptation to novel workload patterns on symmetric workload category representations; and multi-objective reward engineering that balances SLO compliance, resource efficiency, fairness, and stability. The experimental evaluation demonstrates substantial improvements over state-of-the-art methods, achieving 43.5% reduction in p99 latency violations for OLTP workloads and 27.6% improvement in CPU utilization. The

system successfully scales to 10,000 concurrent processes while maintaining scheduling overhead below 3%, validating its practicality for large-scale deployments. Our ablation studies confirm that each component contributes meaningfully to overall performance, with the hierarchical architecture proving most critical by enabling efficient learning through natural problem decomposition.

While our work significantly advances database CPU scheduling, several limitations suggest directions for future research. The current framework focuses exclusively on CPU resources, yet modern database performance depends critically on memory bandwidth, I/O throughput, and network latency—extending HDRL to jointly optimize these resources presents both algorithmic and systems challenges. The four-category workload taxonomy, though covering common scenarios, may require refinement for emerging applications such as graph analytics, time-series processing, or machine learning inference within databases. Future work should explore higher-order symmetries in database architectures, investigate neuromorphic hardware acceleration for real-time symmetric computation, and develop theoretical frameworks that provide symmetric performance guarantees under distribution shift. The assumption of cooperative processes within a single administrative domain requires reconsideration for cloud-native deployments where adversarial behavior and strict isolation requirements prevail. Future work should explore federated learning methods that preserve tenant privacy while enabling cross-workload optimization, investigate neuromorphic hardware acceleration for real-time GNN inference, and develop theoretical frameworks that provide performance guarantees under distribution shift. The vision of fully autonomous database systems requires continued advances in learning-based resource management, and our hierarchical approach provides a foundation for this evolution.

Author Contributions: Methodology, S.X., Y.W. and W.L.; Software, S.X., Y.W. and W.L.; Validation, S.X.; Formal analysis, S.X.; Writing—original draft, S.X.; Writing—review & editing, Y.W. and W.L.; Supervision, Y.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

Nomenclature

State and Action Variables	
s_h	High-level state representation for meta-controller
s_l^c	Low-level state representation for sub-controller of category c
a_h	Meta-controller action (CPU budget allocation)
a_l^c	Sub-controller action for category c (process-level scheduling)
$\mathcal{S}_h, \mathcal{A}_h$	Meta-controller state and action spaces
$\mathcal{S}_l^c, \mathcal{A}_l^c$	Sub-controller state and action spaces for category c
Network Parameters	
θ	Meta-controller policy network parameters
ϕ	Meta-controller value network parameters
ψ	Sub-controller actor network parameters
ξ	Curiosity module dynamics model parameters
W_m, b_m	GNN message function weights and biases
W_a, a	GNN attention mechanism parameters

Performance Metrics	
l_{p99}^t	99th percentile latency at time t
u^t	Global CPU utilization at time t
α_c^t	CPU budget allocation for category c at time t
v_{SLO}^t	SLO violation statistics vector
m_c	Performance metrics vector for workload category c
τ^t	Transaction throughput measurement
Reward Components	
r_{SLO}^t	SLO compliance reward component
r_{util}^t	Resource utilization reward component
r_{fair}^t	Fairness reward component
r_{stable}^t	Stability reward component
$r_{intrinsic}^t$	Curiosity-driven intrinsic reward
w_1, w_2, w_3, w_4	Multi-objective reward weights
Graph Neural Network Variables	
$G_t = (V_t, E_t)$	Process dependency graph at time t
x_i	Node feature vector for process i
e_{ij}	Edge feature vector between processes i and j
$h_i^{(k)}$	Hidden state of node i at layer k
z_{global}	Global graph embedding
z_c	Category-specific graph embedding
System Variables	
$\mathcal{C}$	Set of workload categories {OLTP, OLAP, vec, bg}
T_h, T_l	Meta-controller and sub-controller update periods
α_{min}	Minimum resource allocation threshold
γ_h, γ_l	Discount factors for hierarchical levels
β, κ	Meta-learning and curiosity decay parameters

References

1. Ooi, B.C.; Cai, S.; Chen, G.; Shen, Y.; Tan, K.L.; Wu, Y.; Xiao, X.; Xing, N.; Yue, C.; Zeng, L.; et al. NeurDB: An AI-powered autonomous data system. *Sci. China Inf. Sci.* **2024**, *67*, 200901. [CrossRef]
2. Chrysogelos, P.; Karpathiotakis, M.; Appuswamy, R.; Ailamaki, A. HetExchange: Encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *Proc. Vldb Endow.* **2019**, *12*, 544–556. [CrossRef]
3. Yogatama, B.W.; Gong, W.; Yu, X. Orchestrating data placement and query execution in heterogeneous CPU-GPU DBMS. *Proc. VLDB Endow.* **2022**, *15*, 2491–2503. [CrossRef]
4. Mao, H.; Alizadeh, M.; Menache, I.; Kandula, S. Resource management with deep reinforcement learning. In Proceedings of the 15th ACM Workshop on Hot Topics in Networks, Atlanta, GA, USA, 9–10 November 2016; pp. 50–56.
5. Liu, N.; Li, Z.; Xu, J.; Xu, Z.; Lin, S.; Qiu, Q.; Tang, J.; Wang, Y. A hierarchical framework of cloud resource allocation and power management using deep reinforcement learning. In Proceedings of the 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), Atlanta, GA, USA, 5–8 June 2017; IEEE: New York, NY, USA, 2017; pp. 372–382.
6. Zhao, Z.; Verma, G.; Rao, C.; Swami, A.; Segarra, S. Distributed scheduling using graph neural networks. In Proceedings of the ICASSP 2021–2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Toronto, ON, Canada, 6–11 June 2021; IEEE: New York, NY, USA, 2021; pp. 4720–4724.
7. Tang, D.; Wang, J.; Chen, R.; Wang, L.; Yu, W.; Zhou, J.; Li, K. Xgnn: Boosting multi-gpu gnn training via global gnn memory store. *Proc. VLDB Endow.* **2024**, *17*, 1105–1118. [CrossRef]
8. Pateria, S.; Subagdja, B.; Tan, A.h.; Quek, C. Hierarchical reinforcement learning: A comprehensive survey. *ACM Comput. Surv. (CSUR)* **2021**, *54*, 1–35. [CrossRef]
9. Hu, X.; Zhang, R.; Tang, K.; Guo, J.; Yi, Q.; Chen, R.; Du, Z.; Li, L.; Guo, Q.; Chen, Y.; et al. Causality-driven hierarchical structure discovery for reinforcement learning. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 20064–20076.
10. Nagabandi, A.; Clavera, I.; Liu, S.; Fearing, R.S.; Abbeel, P.; Levine, S.; Finn, C. Learning to adapt in dynamic, real-world environments through meta-reinforcement learning. *arXiv* **2018**, arXiv:1803.11347.
11. Alet, F.; Schneider, M.F.; Lozano-Perez, T.; Kaelbling, L.P. Meta-learning curiosity algorithms. *arXiv* **2020**, arXiv:2003.05325.
12. Jarrett, D.; Tallec, C.; Altché, F.; Mesnard, T.; Munos, R.; Valko, M. Curiosity in hindsight: Intrinsic exploration in stochastic environments. *arXiv* **2022**, arXiv:2211.10515.

13. Li, J.; Gao, H.; Lv, T.; Lu, Y. Deep reinforcement learning based computation offloading and resource allocation for MEC. In Proceedings of the 2018 IEEE Wireless Communications and Networking Conference (WCNC), Barcelona, Spain, 15–18 April 2018; IEEE: New York, NY, USA, 2018; pp. 1–6.
14. Ye, H.; Li, G.Y.; Juang, B.H.F. Deep reinforcement learning based resource allocation for V2V communications. *IEEE Trans. Veh. Technol.* **2019**, *68*, 3163–3173. [CrossRef]
15. Feng, Y.; Liu, F. Resource management in cloud computing using deep reinforcement learning: A survey. In *Proceedings of the China Aeronautical Science and Technology Youth Science Forum*; Springer: Singapore, 2022; pp. 635–643.
16. Chen, Z.; Hu, J.; Min, G.; Luo, C.; El-Ghazawi, T. Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *33*, 1911–1923. [CrossRef]
17. Dong, T.; Xue, F.; Xiao, C.; Zhang, J. Workflow scheduling based on deep reinforcement learning in the cloud environment. *J. Ambient. Intell. Humaniz. Comput.* **2021**, *12*, 10823–10835. [CrossRef]
18. Zhang, J.; Liu, Y.; Zhou, K.; Li, G.; Xiao, Z.; Cheng, B.; Xing, J.; Wang, Y.; Cheng, T.; Liu, L.; et al. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In Proceedings of the 2019 International Conference on Management of Data, Amsterdam, The Netherlands, 30 June–5 July 2019; pp. 415–432.
19. Manczak, B.; Viebahn, J.; van Hoof, H. Hierarchical reinforcement learning for power network topology control. *arXiv* **2023**, arXiv:2311.02129.
20. Li, S.; Wang, R.; Tang, M.; Zhang, C. Hierarchical reinforcement learning with advantage-based auxiliary rewards. *Adv. Neural Inf. Process. Syst.* **2019**, *32*, 1409–1419.
21. Nachum, O.; Gu, S.S.; Lee, H.; Levine, S. Data-efficient hierarchical reinforcement learning. *Adv. Neural Inf. Process. Syst.* **2018**, *31*, 3307–3317.
22. Zhao, Z.; Verma, G.; Swami, A.; Segarra, S. Delay-oriented distributed scheduling using graph neural networks. In Proceedings of the ICASSP 2022–2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Singapore, 23–27 May 2022; IEEE: New York, NY, USA, 2022; pp. 8902–8906.
23. Wang, Q.; Zhang, Y.; Wang, H.; Chen, C.; Zhang, X.; Yu, G. Neutronstar: Distributed GNN training with hybrid dependency management. In Proceedings of the Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA, 12–17 June 2022; pp. 1301–1315.
24. Wang, L.; Yin, Q.; Tian, C.; Yang, J.; Chen, R.; Yu, W.; Yao, Z.; Zhou, J. FlexGraph: A flexible and efficient distributed framework for GNN training. In Proceedings of the Proceedings of the Sixteenth European Conference on Computer Systems, Online Event, 26–18 April 2021; pp. 67–82.
25. Song, Z.; Antsaklis, P.J.; Lin, H. Graph Neural Network-Based Distributed Optimal Control for Linear Networked Systems: An Online Distributed Training Approach. *arXiv* **2025**, arXiv:2504.06439.
26. Nogueira Lobo de Carvalho, M.; Simitsis, A.; Queralt Calafat, A.; Romero Moral, Ó. Workload placement on heterogeneous CPU-GPU systems. *Proc. VLDB Endow.* **2024**, *17*, 4241–4244. [CrossRef]
27. Lyu, C.; Fan, Q.; Song, F.; Sinha, A.; Diao, Y.; Chen, W.; Ma, L.; Feng, Y.; Li, Y.; Zeng, K.; et al. Fine-grained modeling and optimization for intelligent resource management in big data processing. *arXiv* **2022**, arXiv:2207.02026. [CrossRef]
28. El Danaoui, M.; Yin, S.; Hameurlain, A.; Morvan, F. Leveraging Workload Prediction for Query Optimization in Multi-Tenant Parallel DBMSs. In Proceedings of the 2024 8th International Conference on Cloud and Big Data Computing, Oxford, UK, 15–17 August 2024; pp. 41–48.
29. Zhu, R.; Weng, L.; Wei, W.; Wu, D.; Peng, J.; Wang, Y.; Ding, B.; Lian, D.; Zheng, B.; Zhou, J. Pilotscope: Steering databases with machine learning drivers. *Proc. VLDB Endow.* **2024**, *17*, 980–993. [CrossRef]
30. Zhang, X.; Wu, H.; Li, Y.; Tang, Z.; Tan, J.; Li, F.; Cui, B. An efficient transfer learning based configuration adviser for database tuning. *Proc. VLDB Endow.* **2023**, *17*, 539–552. [CrossRef]
31. Zhang, W.; Lim, W.S.; Butrovich, M.; Pavlo, A. The Holon Approach for Simultaneously Tuning Multiple Components in a Self-Driving Database Management System with Machine Learning via Synthesized Proto-Actions. *Proc. VLDB Endow.* **2024**, *17*, 3373–3387. [CrossRef]
32. Lim, W.S.; Ma, L.; Zhang, W.; Butrovich, M.; Arch, S.; Pavlo, A. Hit the Gym: Accelerating Query Execution to Efficiently Bootstrap Behavior Models for Self-Driving Database Management Systems. *Proc. VLDB Endow.* **2024**, *17*, 3680–3693. [CrossRef]
33. Zhu, Y.; Tian, Y.; Cahoon, J.; Krishnan, S.; Agarwal, A.; Alotaibi, R.; Camacho-Rodríguez, J.; Chundatt, B.; Chung, A.; Dutta, N.; et al. Towards building autonomous data services on azure. In Proceedings of the Companion of the 2023 International Conference on Management of Data, Seattle, WA, USA, 18–23 June 2023; pp. 217–224.
34. Beck, J.; Vuorio, R.; Liu, E.Z.; Xiong, Z.; Zintgraf, L.; Finn, C.; Whiteson, S. A survey of meta-reinforcement learning. *arXiv* **2023**, arXiv:2301.08028.
35. Aubret, A.; Matignon, L.; Hassas, S. An information-theoretic perspective on intrinsic motivation in reinforcement learning: A survey. *Entropy* **2023**, *25*, 327. [CrossRef] [PubMed]
36. Yuan, M. Intrinsically-motivated reinforcement learning: A brief introduction. *arXiv* **2022**, arXiv:2203.02298.

37. Raileanu, R.; Rocktäschel, T. Ride: Rewarding impact-driven exploration for procedurally-generated environments. *arXiv* **2020**, arXiv:2002.12292.
38. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347.
39. Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; PMLR: New York, NY, USA, 2018; pp. 1861–1870.
40. Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized experience replay. *arXiv* **2015**, arXiv:1511.05952.
41. Kipf, T.N.; Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
42. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Message passing neural networks. In *Machine Learning Meets Quantum Physics*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 199–214.
43. Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv* **2015**, arXiv:1506.02438.
44. Pabla, C.S. Completely fair scheduler. *Linux J.* **2009**, *2009*, 4.
45. Corporation, O. Managing Resources with Oracle Database Resource Manager. In *Oracle Database Administrator's Guide, 23c*; Oracle Corporation: Redwood City, CA, USA, 2023.
46. Agrawal, S.; Goyal, N. Thompson sampling for contextual bandits with linear payoffs. In Proceedings of the 30th International Conference on International Conference on Machine Learning, Atlanta, GA, USA, 16–21 June 2013; PMLR: New York, NY, USA, 2013; pp. 127–135.
47. Belgacem, A.; Mahmoudi, S.; Kihl, M. Intelligent multi-agent reinforcement learning model for resources allocation in cloud computing. *J. King Saud-Univ.-Comput. Inf. Sci.* **2022**, *34*, 2391–2404. [CrossRef]
48. Cheong, M.; Lee, H.; Yeom, I.; Woo, H. SCARL: Attentive reinforcement learning-based scheduling in a multi-resource heterogeneous cluster. *IEEE Access* **2019**, *7*, 153432–153444. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.